

Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет програмних систем та прикладної математики
Кафедра системного програмування і спеціалізованих комп’ютерних систем

ЛАБОРАТОРНА РОБОТА №»2

з дисципліни

“Розробка програмного забезпечення систем реального часу”

Тема: «Комунікаційні інтерфейси та якість обслуговування QoS»

Виконав: Ляшенко Д.Д.
Студент групи КВ-33

Київ 2026

Мета лабораторної роботи:

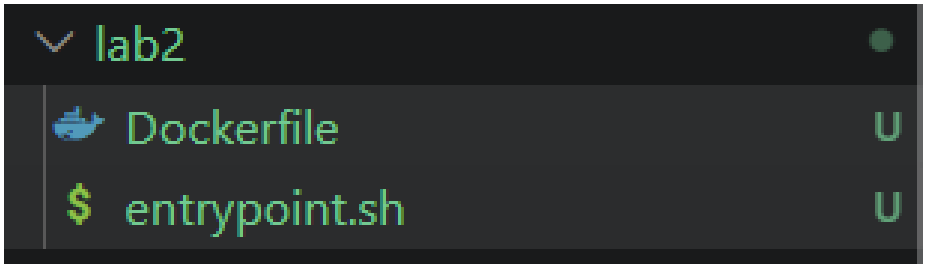
Навчитися проектувати топіки, асинхронні сервіси та дії (Actions) в ROS2, конфігурувати сумісні політики QoS (Quality of Service), реалізовувати “ледачу” публікацію даних (Lazy Publishers) та оптимізовувати обмін даними за допомогою механізму адаптації типів (Type Adaptation).

Порядок виконання лабораторної роботи:

Необхідно створити вузл обробки та фільтрації даних data_filter:

1. Написати C++ вузол data_filter, який підписуєтьсчя на вхідну тему з рядками std_msgs::msg::String, переводить текст у верхній регістр та публікує його у вихідну тему uppercase_topic.
2. Реалізувати matched_callback для вихідного видавця: підписка на вхідні дані має створюватися лише тоді, коли з’являється хоча б один слухач на uppercase_topic, і вимикається, коли кількість слухачів стає 0.
3. Конфігурувати видавця з використанням QoS профілю transient_local глибиною 1 щоб передавати тільки останнє повідомлення.
4. Додати підтримку перевизначення політик Depth, Reliability, Durability за допомогою rclpp::QosOverridingOptions.
5. Реалізувати асинхронний сервіс та його інтросекції: Додати до вузла сервісний сервер ResetFilter. Написати сервісний клієнт, що виконує асинхронний виклик async_send_request з обробкою таймауту очікування через future_wait_for(…). Ввімкнути інтроспекцію сервісу через configure_introspection (зі станом RCL_SERVICE_INTROSPECTION_CONTENTS) та перевірити перехоплення подій через CLI-команди з опцією “include_hidden_topics”.
6. Розробити Action-сервер, реалізувати асинхронне виконання обробки масиву даних у окремому потоці, публікацію періодичного фідбеку про поточний стан (чи досягнутий стан Succeded).
7. Створити адаптер між типом msg/PointCloud2 Documentation і будь-яким на ваш вибір з бібліотеки за посиланням. Створити видавця та підписника використовуючи цей функціонал у різних компонентах, і переконайтеся, що перетворення даних не виконується, якщо ввімкнено внутрішньопроцесорну взаємодію.

Хід виконання лабораторної роботи:



Dockerfile U

lab2 > Dockerfile

```
1  FROM osrf/ros:jazzy-desktop
2
3  COPY entrypoint.sh /entrypoint.sh
4  RUN chmod +x /entrypoint.sh
5
6  WORKDIR /ros2_ws
7  ENTRYPOINT ["/entrypoint.sh"]
8  CMD ["bash"]
```

Windows PowerShell

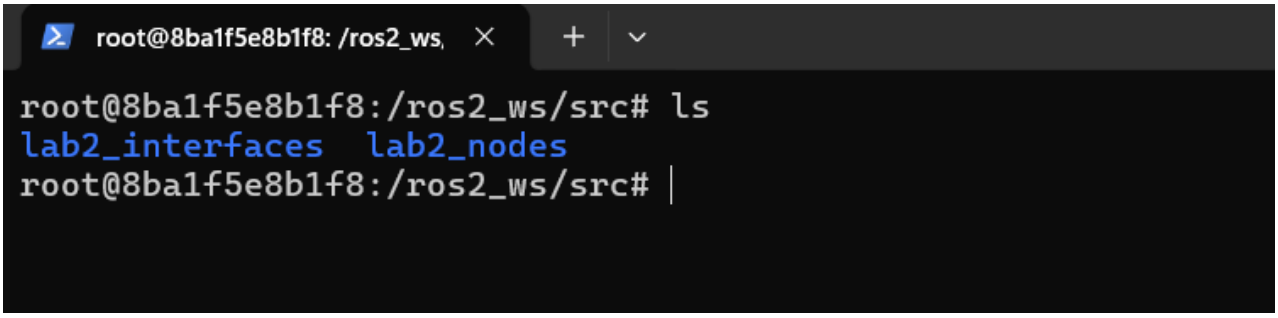
```
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker build -t my_ros2_jazzy .
[+] Building 507.0s (9/9) FINISHED                                              docker:desktop-linux
 => [internal] load build definition from Dockerfile                                          0.1s
```

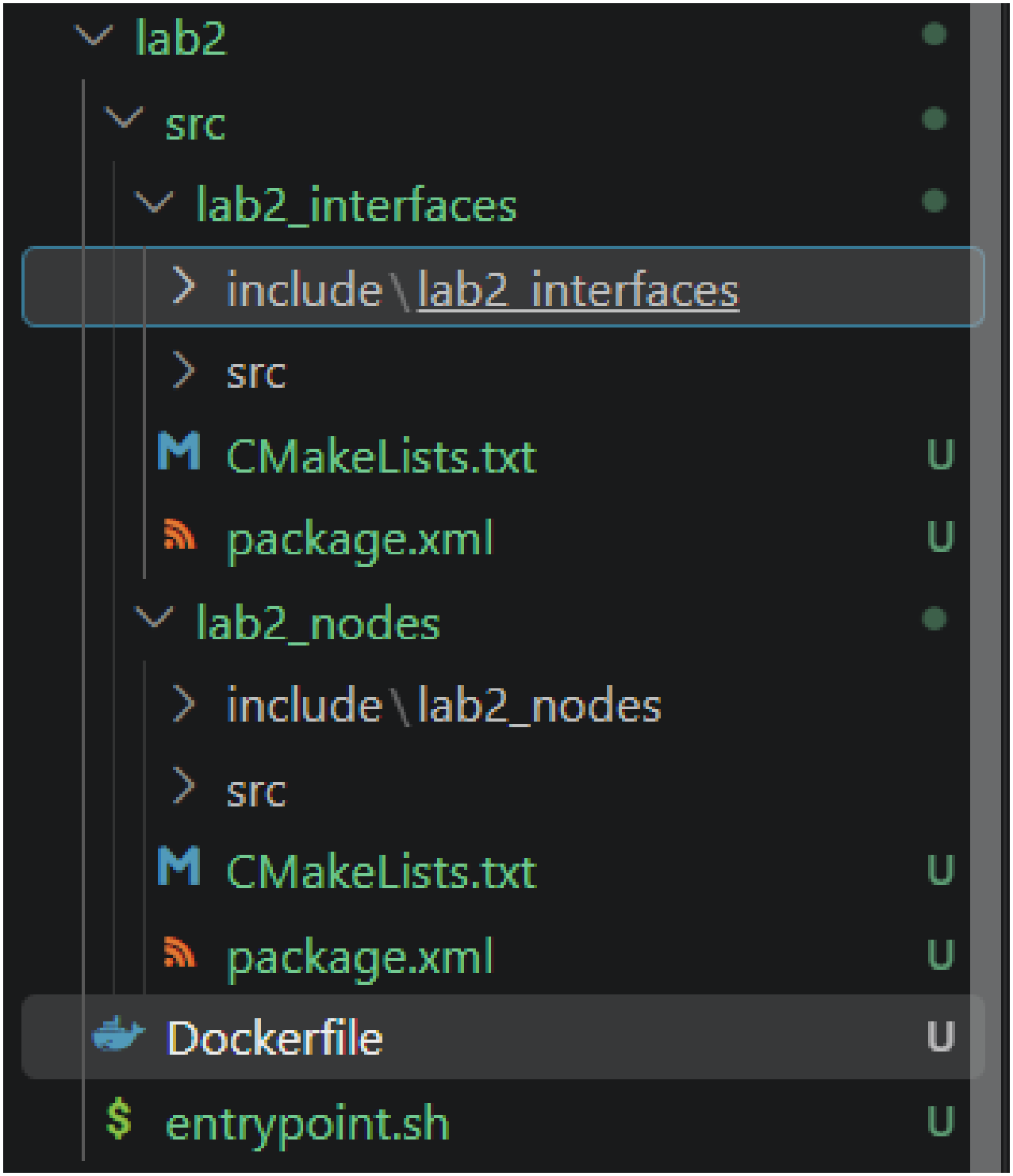
root@8ba1f5e8b1f8: /ros2_ws

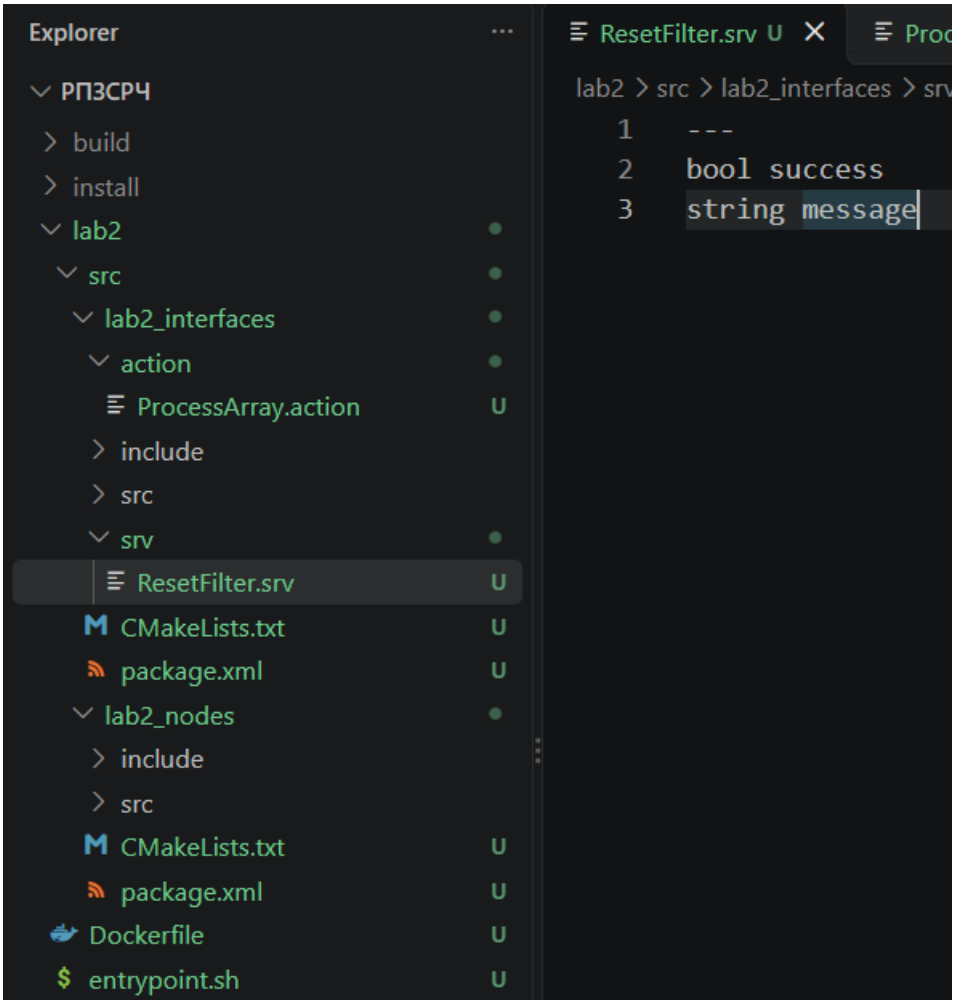
```
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker run -it -v ${PWD}:/ros2_ws my_ros2_jazzy
root@8ba1f5e8b1f8:/ros2_ws# echo $ROS_DISTRO
jazzy
root@8ba1f5e8b1f8:/ros2_ws# mkdir -p src
cd src
ros2 pkg create --build-type ament_cmake lab2_interfaces
ros2 pkg create --build-type ament_cmake lab2_nodes --dependencies rclcpp rclcpp_action rclcpp_components std_msgs lab2_interfaces
going to create a new package
package name: lab2_interfaces
destination directory: /ros2_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['root <root@todo.todo>']
licenses: ['TODO: License declaration']
build type: ament_cmake
dependencies: []
creating folder ./lab2_interfaces
creating ./lab2_interfaces/package.xml
creating source and include folder
creating folder ./lab2_interfaces/src
creating folder ./lab2_interfaces/include/lab2_interfaces
creating ./lab2_interfaces/CMakeLists.txt
```

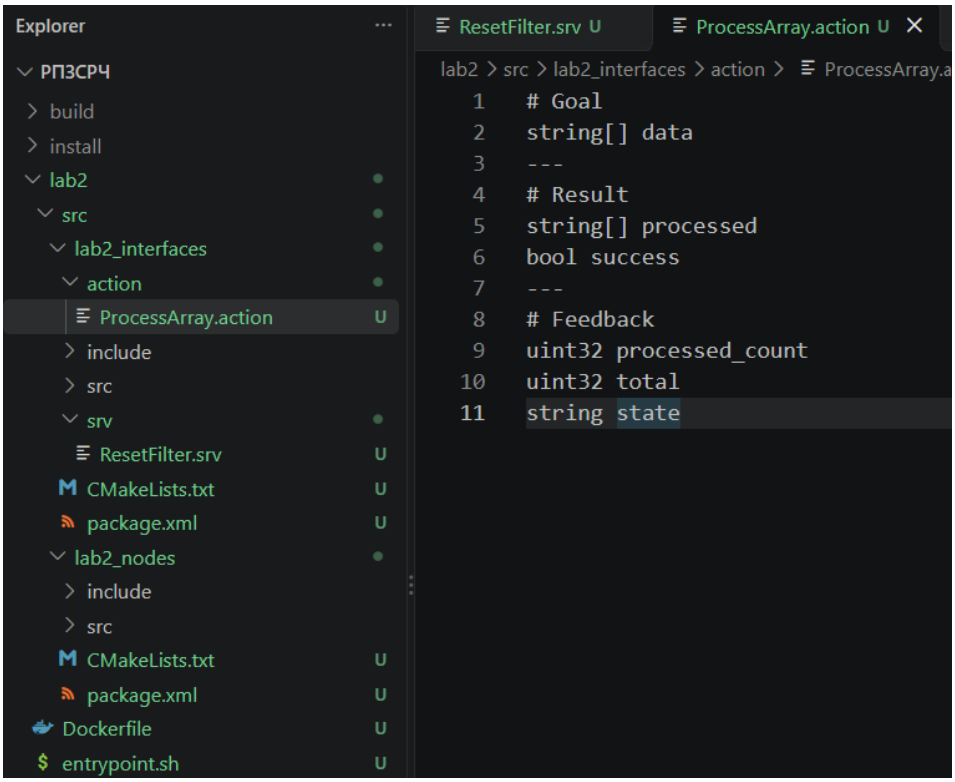
```
[WARNING]: Unknown license 'TODO: License declaration'.  This has been set in the package.xml, but no LICENSE file has been created.
It is recommended to use one of the ament license identifiers:
Apache-2.0
BSL-1.0
BSD-2.0
BSD-2-Clause
BSD-3-Clause
GPL-3.0-only
LGPL-3.0-only
MIT
MIT-0
going to create a new package
package name: lab2_nodes
destination directory: /ros2_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['root <root@todo.todo>']
licenses: ['TODO: License declaration']
build type: ament_cmake
dependencies: ['rclcpp', 'rclcpp_action', 'rclcpp_components', 'std_msgs', 'lab2_interfaces']
creating folder ./lab2_nodes
creating ./lab2_nodes/package.xml
creating source and include folder
creating folder ./lab2_nodes/src
creating folder ./lab2_nodes/include/lab2_nodes
creating ./lab2_nodes/CMakeLists.txt
```

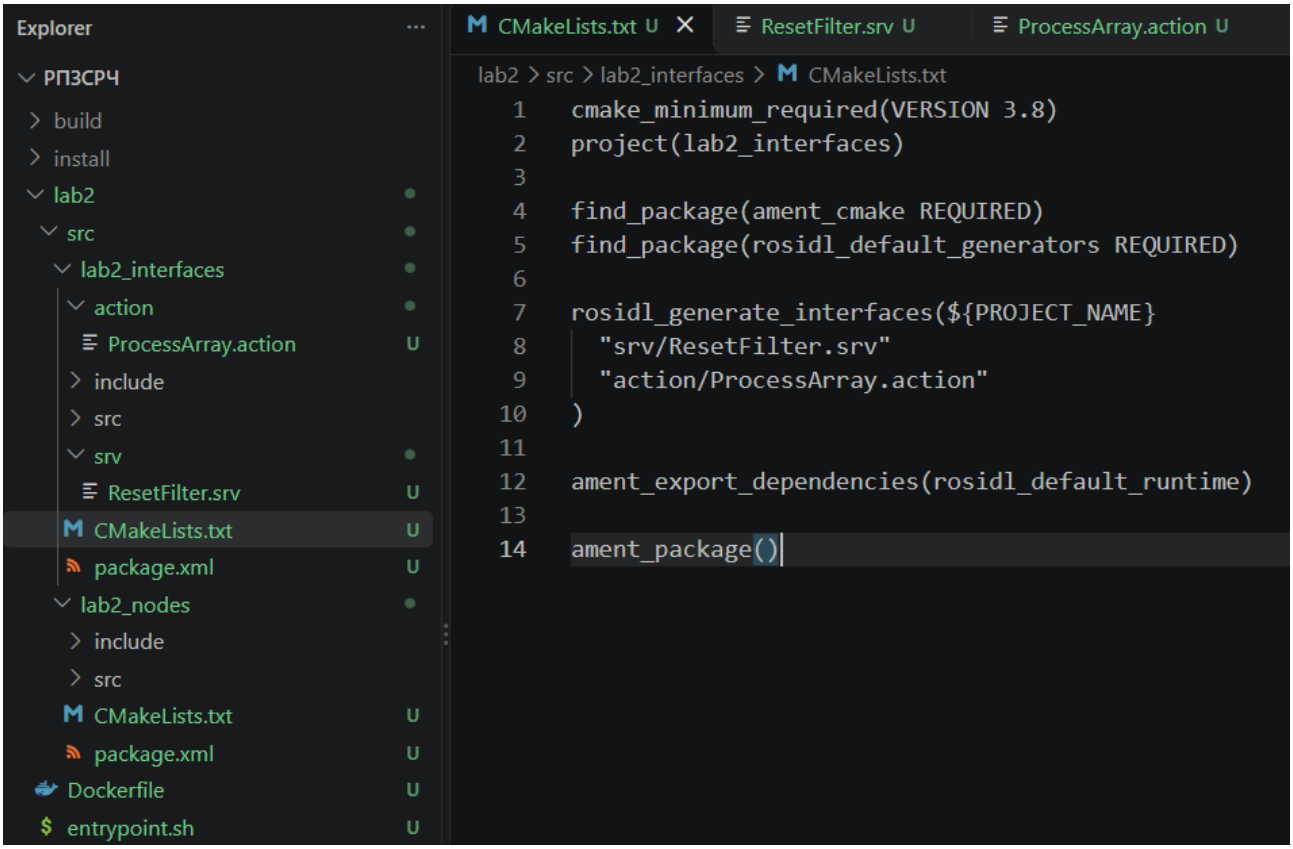
```
[WARNING]: Unknown license 'TODO: License declaration'.  This has been set in the package.xml, but no LICENSE file has been created.
It is recommended to use one of the ament license identifiers:
Apache-2.0
BSL-1.0
BSD-2.0
BSD-2-Clause
BSD-3-Clause
GPL-3.0-only
LGPL-3.0-only
MIT
MIT-0
root@8ba1f5e8b1f8:/ros2_ws/src#
```

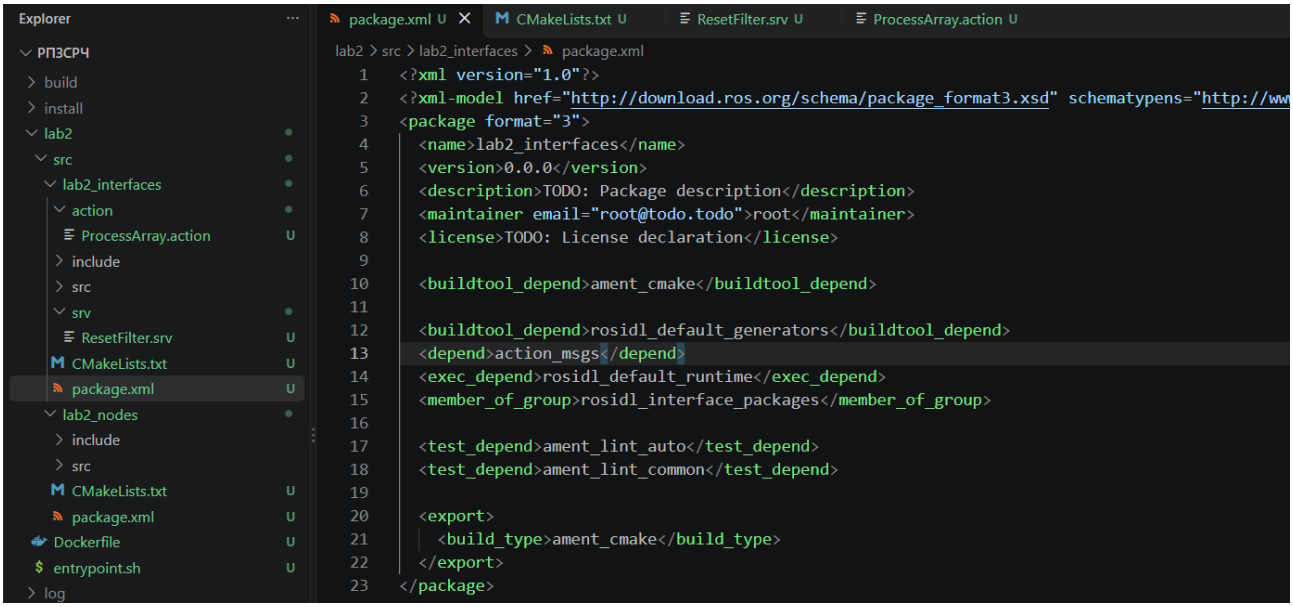

root@8ba1f5e8b1f8: /ros2_ws
root@8ba1f5e8b1f8:/ros2_ws/src# ls
lab2_interfaces lab2_nodes
root@8ba1f5e8b1f8:/ros2_ws/src#


lab2
src
lab2_interfaces
include\lab2_interfaces
src
CMakeLists.txt U
package.xml U
lab2_nodes
include\lab2_nodes
src
CMakeLists.txt U
package.xml U
Dockerfile U
entrypoint.sh U


Explorer
РПЗСРЧ
build
install
lab2
src
lab2_interfaces
action
ProcessArray.action U
include
src
srv
ResetFilter.srv U
CMakeLists.txt U
package.xml U
lab2_nodes
include
src
CMakeLists.txt U
package.xml U
Dockerfile U
entrypoint.sh U
ResetFilter.srv U
lab2 > src > lab2_interfaces > srv
1 ---
2 bool success
3 string message

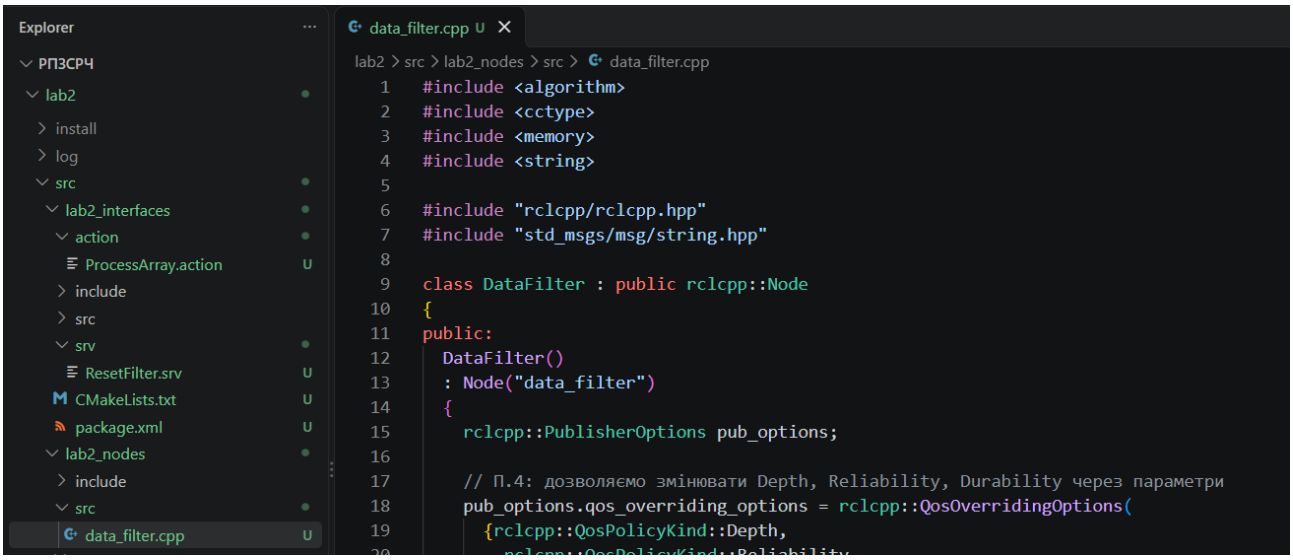

Explorer
РПЗСРЧ
build
install
lab2
src
lab2_interfaces
action
ProcessArray.action U
include
src
srv
ResetFilter.srv U
CMakeLists.txt U
package.xml U
lab2_nodes
include
src
CMakeLists.txt U
package.xml U
Dockerfile U
entrypoint.sh U
ResetFilter.srv U
ProcessArray.action U
lab2 > src > lab2_interfaces > action > ProcessArray.a
1 # Goal
2 string[] data
3 ---
4 # Result
5 string[] processed
6 bool success
7 ---
8 # Feedback
9 uint32 processed_count
10 uint32 total
11 string state


Explorer
РПЗСРЧ
build
install
lab2
src
lab2_interfaces
action
ProcessArray.action U
include
src
srv
ResetFilter.srv U
CMakeLists.txt U
package.xml U
lab2_nodes
include
src
CMakeLists.txt U
package.xml U
Dockerfile U
entrypoint.sh U
CMakeLists.txt U
ResetFilter.srv U
ProcessArray.action U
lab2 > src > lab2_interfaces > CMakeLists.txt
cmake_minimum_required(VERSION 3.8)
project(lab2_interfaces)
find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)
rosidl_generate_interfaces(${PROJECT_NAME}
"srv/ResetFilter.srv"
"action/ProcessArray.action"
)
ament_export_dependencies(rosidl_default_runtime)
ament_package()


Explorer
РПЗСРЧ
build
install
lab2
src
lab2_interfaces
action
ProcessArray.action U
include
src
srv
ResetFilter.srv U
CMakeLists.txt U
package.xml U
lab2_nodes
include
src
CMakeLists.txt U
package.xml U
Dockerfile U
entrypoint.sh U
log
package.xml U
CMakeLists.txt U
ResetFilter.srv U
ProcessArray.action U
lab2 > src > lab2_interfaces > package.xml
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://ww
<package format="3">
<name>lab2_interfaces</name>
<version>0.0.0</version>
<description>TODO: Package description</description>
<maintainer email="root@todo.todo">root</maintainer>
<license>TODO: License declaration</license>
<buildtool_depend>ament_cmake</buildtool_depend>
<buildtool_depend>rosidl_default_generators</buildtool_depend>
<depend>action_msgs</depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
<test_depend>ament_lint_auto</test_depend>
<test_depend>ament_lint_common</test_depend>
<export>
<build_type>ament_cmake</build_type>
</export>
</package>

```
root@8ba1f5e8b1f8:/ros2_ws/src# cd /ros2_ws
colcon build --symlink-install --packages-select lab2_interfaces
source install/setup.bash
Starting >>> lab2_interfaces
[Processing: lab2_interfaces]
Finished <<< lab2_interfaces [44.6s]

Summary: 1 package finished [45.0s]
root@8ba1f5e8b1f8:/ros2_ws# ros2 interface show lab2_interfaces/srv/ResetFilter
ros2 interface show lab2_interfaces/action/ProcessArray
---
bool success
string message
# Goal
string[] data
---
# Result
string[] processed
bool success
---
# Feedback
uint32 processed_count
uint32 total
string state
root@8ba1f5e8b1f8:/ros2_ws#
```




```cpp
#include <algorithm>
#include <cctype>
#include <memory>
#include <string>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

class DataFilter : public rclcpp::Node
{
public:
  DataFilter()
  : Node("data_filter")
  {
    rclcpp::PublisherOptions pub_options;
```

```
    // П.4: дозволяємо змінювати Depth, Reliability, Durability через
параметри
    pub_options.qos_overriding_options = rclcpp::QosOverridingOptions(
      {rclcpp::QosPolicyKind::Depth,
        rclcpp::QosPolicyKind::Reliability,
        rclcpp::QosPolicyKind::Durability});

    // П.2: matched_callback спрацьовує, коли змінюється кількість слухачів
    pub_options.event_callbacks.matched_callback =
      [this](auto & status) {
        on_matched(static_cast<int>(status.current_count));
      };

    // П.3: transient_local, depth = 1 (пізній підписник отримає лише
останнє повідомлення)
    auto qos = rclcpp::QoS(rclcpp::KeepLast(1)).transient_local();

    pub_ = create_publisher<std_msgs::msg::String>("uppercase_topic", qos,
pub_options);

    RCLCPP_INFO(get_logger(), "data_filter запущено, слухачів немає,
підписку не створено");
  }

private:
  void on_matched(int listeners)
  {
    RCLCPP_INFO(get_logger(), "Кількість слухачів uppercase_topic: %d",
listeners);

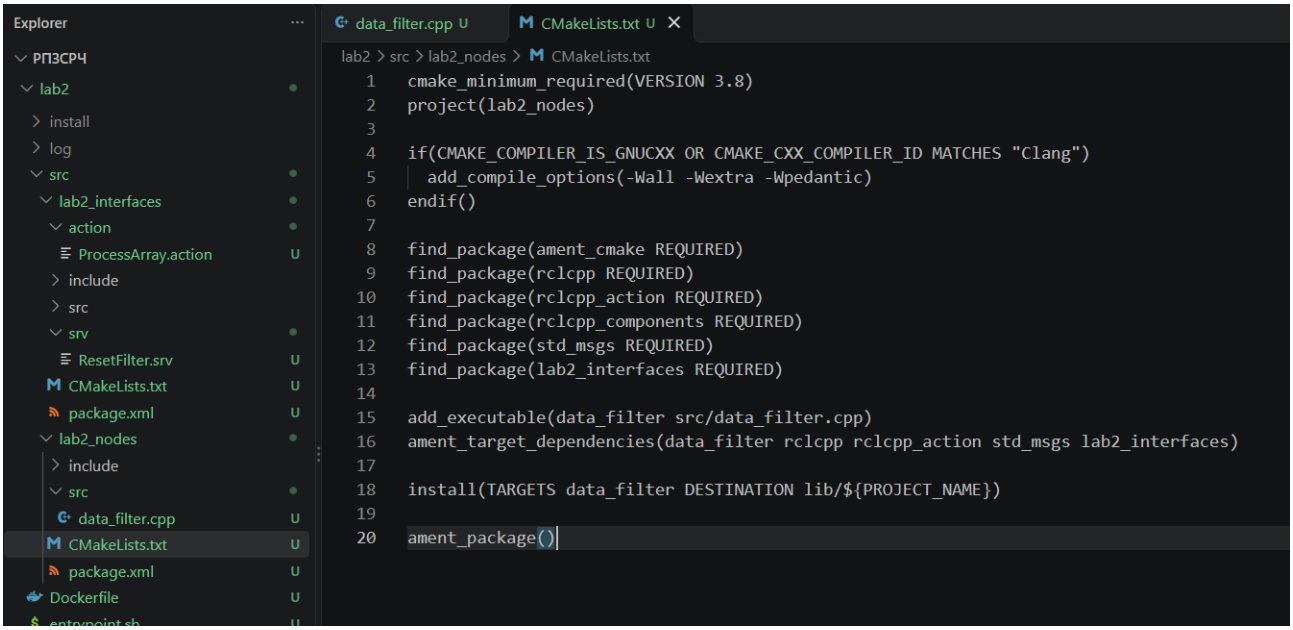
    if (listeners > 0 && !sub_) {
      // З'явився слухач: створюємо підписку на вхідні дані
      sub_ = create_subscription<std_msgs::msg::String>(
        "input_topic", 10,
        [this](const std_msgs::msg::String & msg) { on_input(msg); });
      RCLCPP_INFO(get_logger(), "Підписку на input_topic СТВОРЕНО");
    } else if (listeners == 0 && sub_) {
      // Слухачів не лишилось: прибираємо підписку
      sub_.reset();
      RCLCPP_INFO(get_logger(), "Підписку на input_topic ВИМКНЕНО");
    }
  }

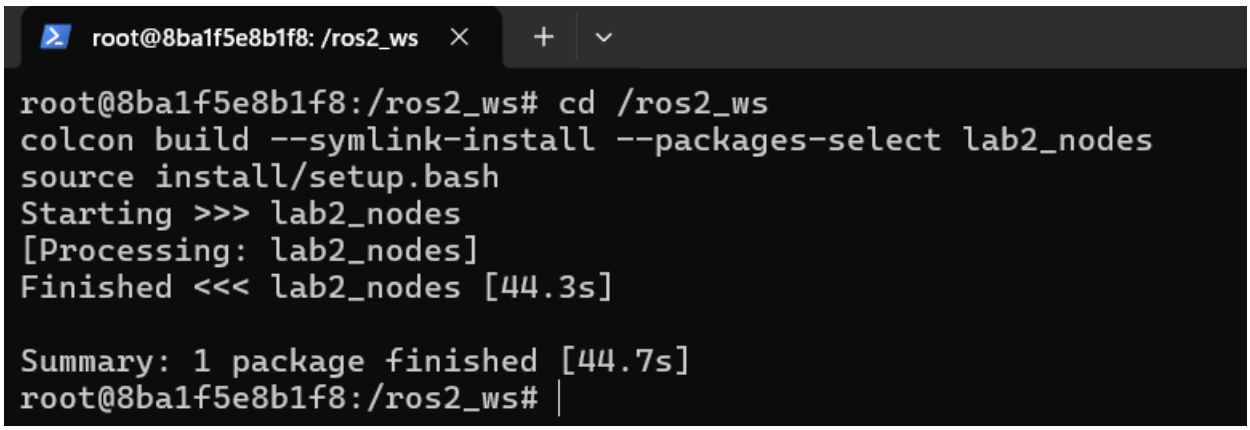
  // П.1: переводимо текст у верхній регістр і публікуємо
  void on_input(const std_msgs::msg::String & msg)
  {
    std_msgs::msg::String out;
    out.data = msg.data;
    std::transform(
      out.data.begin(), out.data.end(), out.data.begin(),
```

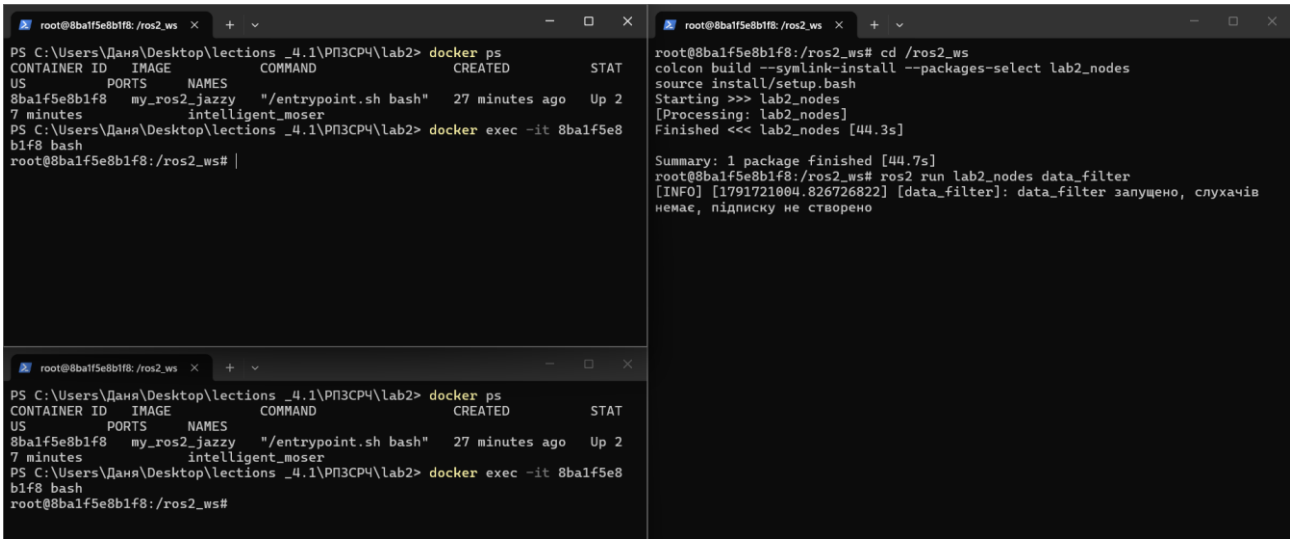
```
      [](unsigned char c) {return static_cast<char>(std::toupper(c));});
    pub_->publish(out);
  }

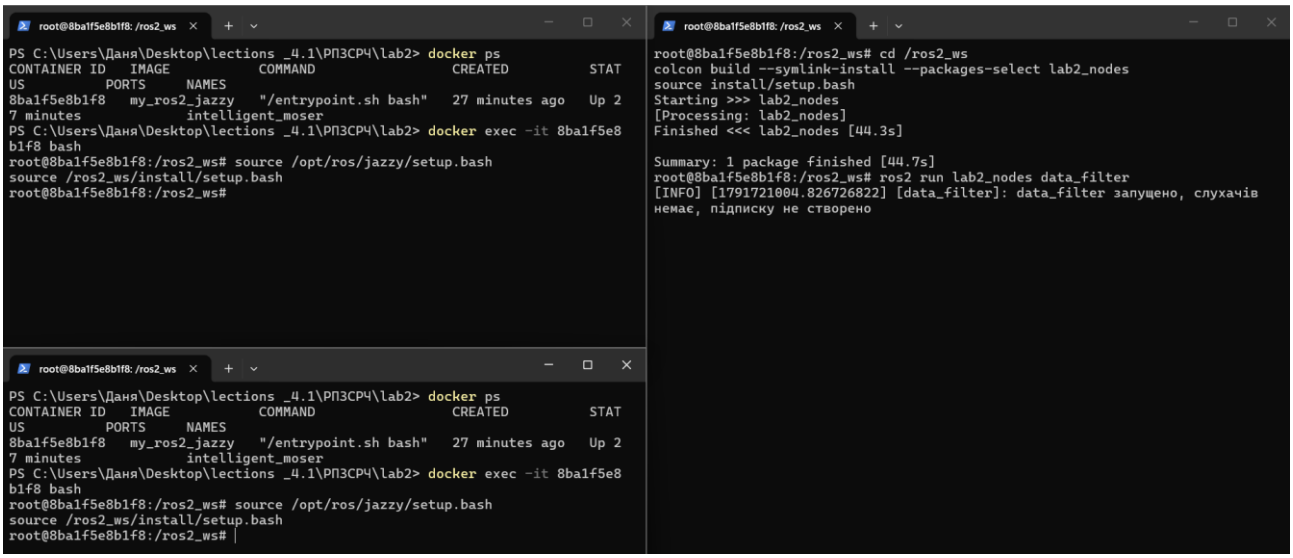
  rclcpp::Publisher<std_msgs::msg::String>::SharedPtr pub_;
  rclcpp::Subscription<std_msgs::msg::String>::SharedPtr sub_;
};

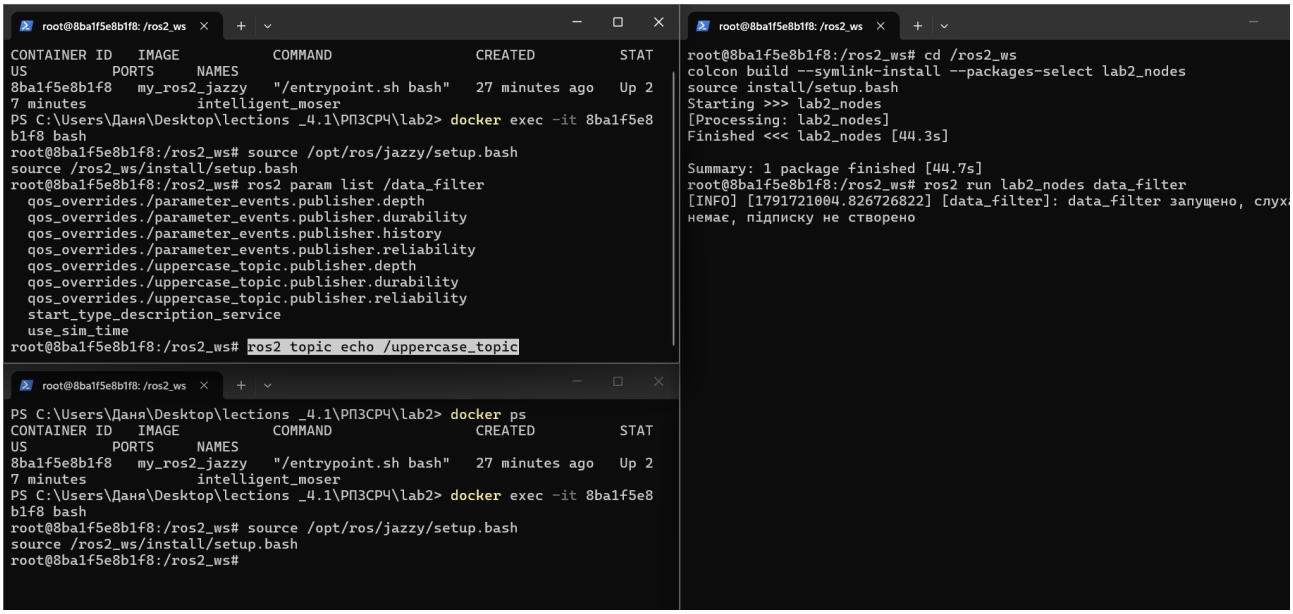
int main(int argc, char ** argv)
{
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<DataFilter>());
  rclcpp::shutdown();
  return 0;
}
```

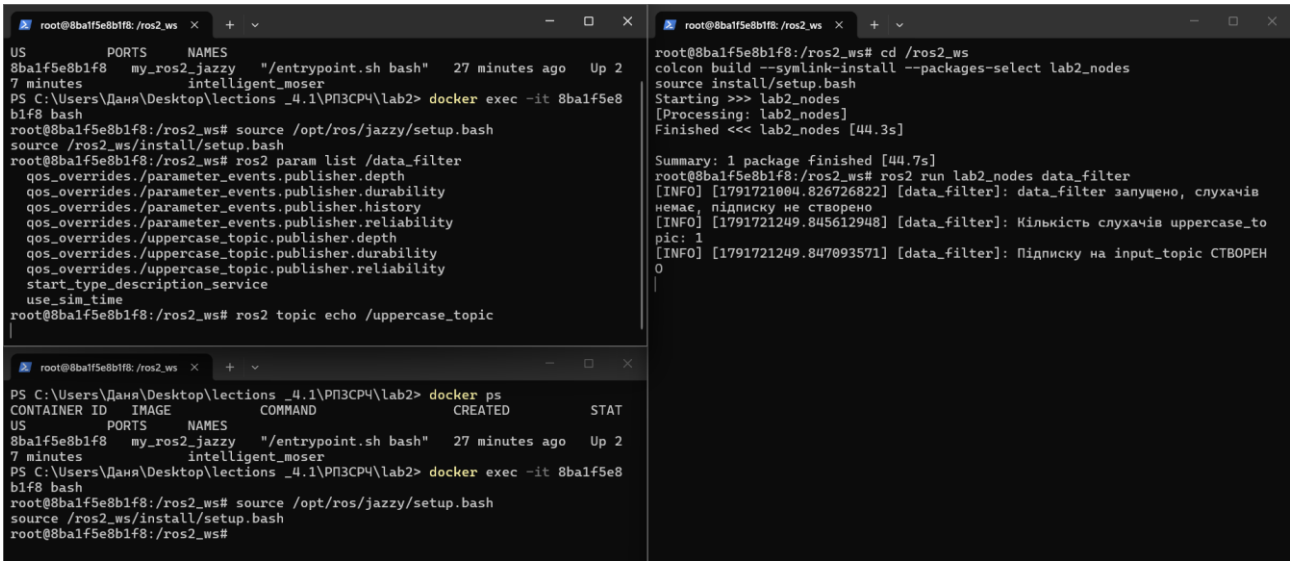


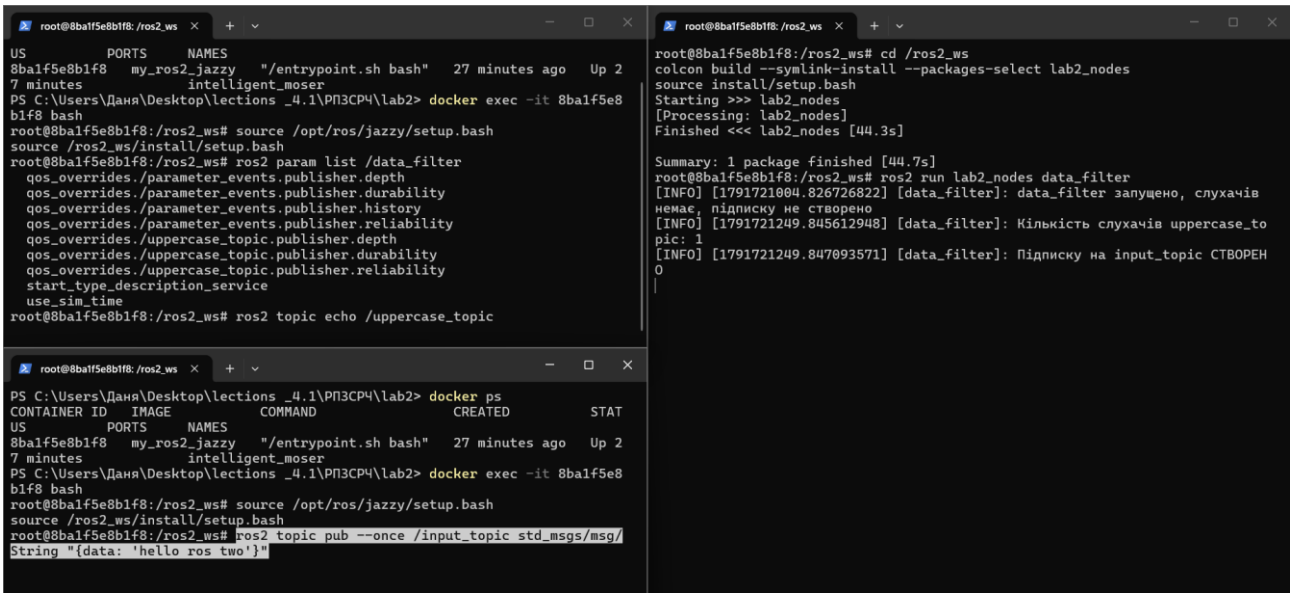


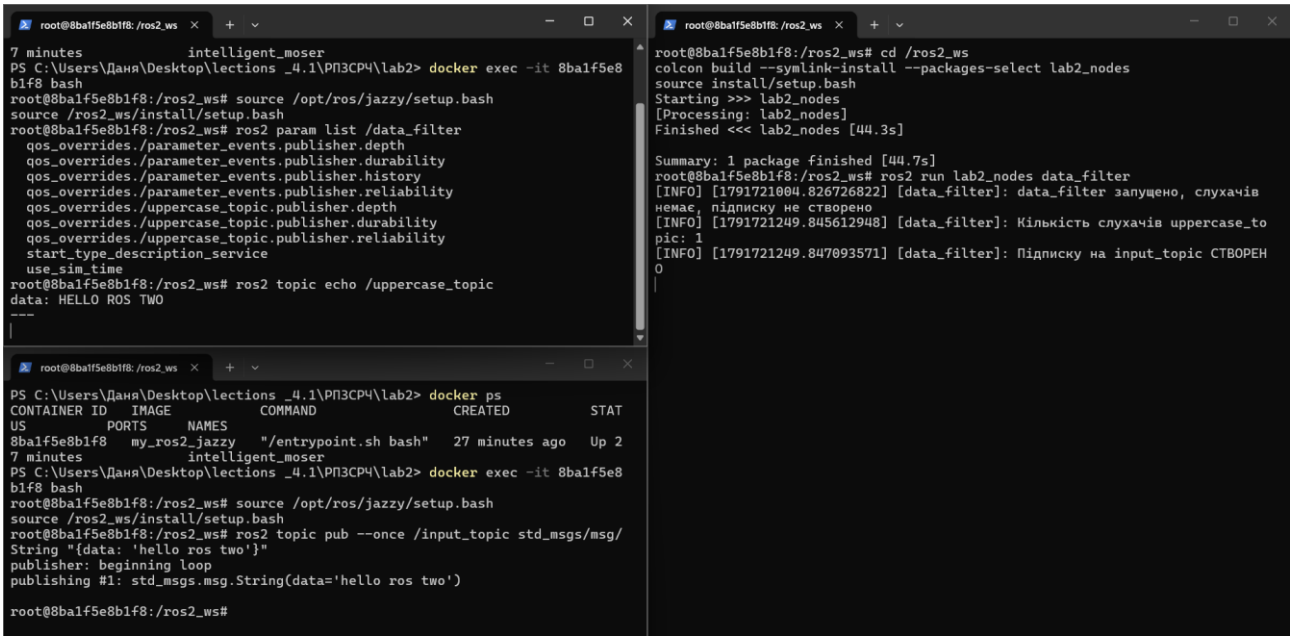

root@8ba1f5e8b1f8: /ros2_ws
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker ps
CONTAINER ID IMAGE COMMAND CREATED STAT
US PORTS NAMES
8ba1f5e8b1f8 my_ros2_jazzy "/entrypoint.sh bash" 27 minutes ago Up 2
7 minutes intelligent_moser
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker exec -it 8ba1f5e8
b1f8 bash
root@8ba1f5e8b1f8:/ros2_ws#
root@8ba1f5e8b1f8: /ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# cd /ros2_ws
colcon build --symlink-install --packages-select lab2_nodes
source install/setup.bash
Starting >>> lab2_nodes
[Processing: lab2_nodes]
Finished <<< lab2_nodes [44.3s]
Summary: 1 package finished [44.7s]
root@8ba1f5e8b1f8:/ros2_ws# ros2 run lab2_nodes data_filter
[INFO] [1791721004.826726822] [data_filter]: data_filter запущено, слухачів
немає, підписку не створено

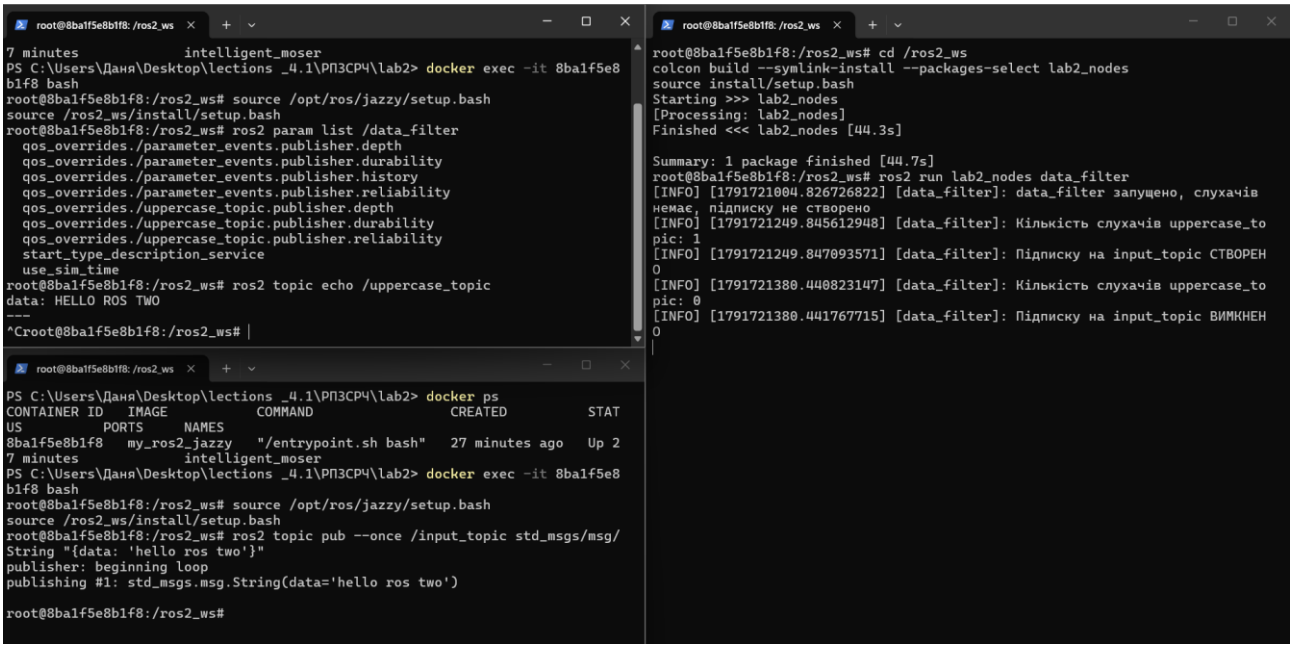

root@8ba1f5e8b1f8: /ros2_ws
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker ps
CONTAINER ID IMAGE COMMAND CREATED STAT
US PORTS NAMES
8ba1f5e8b1f8 my_ros2_jazzy "/entrypoint.sh bash" 27 minutes ago Up 2
7 minutes intelligent_moser
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker exec -it 8ba1f5e8
b1f8 bash
root@8ba1f5e8b1f8:/ros2_ws# source /opt/ros/jazzy/setup.bash
source /ros2_ws/install/setup.bash
root@8ba1f5e8b1f8:/ros2_ws#
root@8ba1f5e8b1f8: /ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# cd /ros2_ws
colcon build --symlink-install --packages-select lab2_nodes
source install/setup.bash
Starting >>> lab2_nodes
[Processing: lab2_nodes]
Finished <<< lab2_nodes [44.3s]
Summary: 1 package finished [44.7s]
root@8ba1f5e8b1f8:/ros2_ws# ros2 run lab2_nodes data_filter
[INFO] [1791721004.826726822] [data_filter]: data_filter запущено, слухачів
немає, підписку не створено

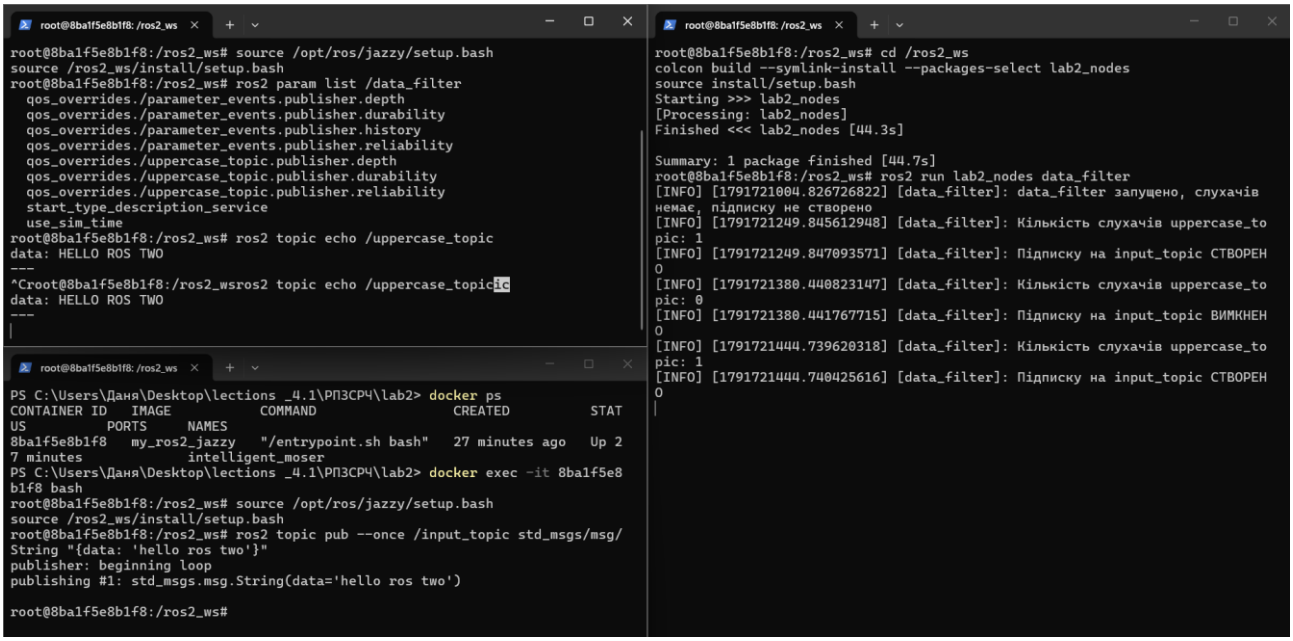

root@8ba1f5e8b1f8: /ros2_ws
CONTAINER ID IMAGE COMMAND CREATED STAT
US PORTS NAMES
8ba1f5e8b1f8 my_ros2_jazzy "/entrypoint.sh bash" 27 minutes ago Up 2
7 minutes intelligent_moser
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker exec -it 8ba1f5e8
b1f8 bash
root@8ba1f5e8b1f8:/ros2_ws# source /opt/ros/jazzy/setup.bash
source /ros2_ws/install/setup.bash
root@8ba1f5e8b1f8:/ros2_ws# ros2 param list /data_filter
qos_overrides./parameter_events.publisher.depth
qos_overrides./parameter_events.publisher.durability
qos_overrides./parameter_events.publisher.history
qos_overrides./parameter_events.publisher.reliability
qos_overrides./uppercase_topic.publisher.depth
qos_overrides./uppercase_topic.publisher.durability
qos_overrides./uppercase_topic.publisher.reliability
start_type_description_service
use_sim_time
root@8ba1f5e8b1f8:/ros2_ws# ros2 topic echo /uppercase_topic
root@8ba1f5e8b1f8: /ros2_ws
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker ps
CONTAINER ID IMAGE COMMAND CREATED STAT
US PORTS NAMES
8ba1f5e8b1f8 my_ros2_jazzy "/entrypoint.sh bash" 27 minutes ago Up 2
7 minutes intelligent_moser
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker exec -it 8ba1f5e8
b1f8 bash
root@8ba1f5e8b1f8:/ros2_ws# source /opt/ros/jazzy/setup.bash
source /ros2_ws/install/setup.bash
root@8ba1f5e8b1f8:/ros2_ws#
root@8ba1f5e8b1f8: /ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# cd /ros2_ws
colcon build --symlink-install --packages-select lab2_nodes
source install/setup.bash
Starting >>> lab2_nodes
[Processing: lab2_nodes]
Finished <<< lab2_nodes [44.3s]
Summary: 1 package finished [44.7s]
root@8ba1f5e8b1f8:/ros2_ws# ros2 run lab2_nodes data_filter
[INFO] [1791721004.826726822] [data_filter]: data_filter запущено, слух
немає, підписку не створено


root@8ba1f5e8b1f8: /ros2_ws
US PORTS NAMES
8ba1f5e8b1f8 my_ros2_jazzy "/entrypoint.sh bash" 27 minutes ago Up 2
7 minutes intelligent_moser
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker exec -it 8ba1f5e8
b1f8 bash
root@8ba1f5e8b1f8:/ros2_ws# source /opt/ros/jazzy/setup.bash
source /ros2_ws/install/setup.bash
root@8ba1f5e8b1f8:/ros2_ws# ros2 param list /data_filter
qos_overrides./parameter_events.publisher.depth
qos_overrides./parameter_events.publisher.durability
qos_overrides./parameter_events.publisher.history
qos_overrides./parameter_events.publisher.reliability
qos_overrides./uppercase_topic.publisher.depth
qos_overrides./uppercase_topic.publisher.durability
qos_overrides./uppercase_topic.publisher.reliability
start_type_description_service
use_sim_time
root@8ba1f5e8b1f8:/ros2_ws# ros2 topic echo /uppercase_topic
root@8ba1f5e8b1f8: /ros2_ws
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker ps
CONTAINER ID IMAGE COMMAND CREATED STAT
US PORTS NAMES
8ba1f5e8b1f8 my_ros2_jazzy "/entrypoint.sh bash" 27 minutes ago Up 2
7 minutes intelligent_moser
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker exec -it 8ba1f5e8
b1f8 bash
root@8ba1f5e8b1f8:/ros2_ws# source /opt/ros/jazzy/setup.bash
source /ros2_ws/install/setup.bash
root@8ba1f5e8b1f8:/ros2_ws#
root@8ba1f5e8b1f8: /ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# cd /ros2_ws
colcon build --symlink-install --packages-select lab2_nodes
source install/setup.bash
Starting >>> lab2_nodes
[Processing: lab2_nodes]
Finished <<< lab2_nodes [44.3s]
Summary: 1 package finished [44.7s]
root@8ba1f5e8b1f8:/ros2_ws# ros2 run lab2_nodes data_filter
[INFO] [1791721004.826726822] [data_filter]: data_filter запущено, слухачів
немає, підписку не створено
[INFO] [1791721249.845612948] [data_filter]: Кількість слухачів uppercase_to
pic: 1
[INFO] [1791721249.847093571] [data_filter]: Підписку на input_topic СТВОРЕН
О


root@8ba1f5e8b1f8: /ros2_ws
US PORTS NAMES
8ba1f5e8b1f8 my_ros2_jazzy "/entrypoint.sh bash" 27 minutes ago Up 2
7 minutes intelligent_moser
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker exec -it 8ba1f5e8
b1f8 bash
root@8ba1f5e8b1f8:/ros2_ws# source /opt/ros/jazzy/setup.bash
source /ros2_ws/install/setup.bash
root@8ba1f5e8b1f8:/ros2_ws# ros2 param list /data_filter
qos_overrides./parameter_events.publisher.depth
qos_overrides./parameter_events.publisher.durability
qos_overrides./parameter_events.publisher.history
qos_overrides./parameter_events.publisher.reliability
qos_overrides./uppercase_topic.publisher.depth
qos_overrides./uppercase_topic.publisher.durability
qos_overrides./uppercase_topic.publisher.reliability
start_type_description_service
use_sim_time
root@8ba1f5e8b1f8:/ros2_ws# ros2 topic echo /uppercase_topic
root@8ba1f5e8b1f8: /ros2_ws
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker ps
CONTAINER ID IMAGE COMMAND CREATED STAT
US PORTS NAMES
8ba1f5e8b1f8 my_ros2_jazzy "/entrypoint.sh bash" 27 minutes ago Up 2
7 minutes intelligent_moser
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker exec -it 8ba1f5e8
b1f8 bash
root@8ba1f5e8b1f8:/ros2_ws# source /opt/ros/jazzy/setup.bash
source /ros2_ws/install/setup.bash
root@8ba1f5e8b1f8:/ros2_ws# ros2 topic pub --once /input_topic std_msgs/msg/
String "{data: 'hello ros two'}"
root@8ba1f5e8b1f8: /ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# cd /ros2_ws
colcon build --symlink-install --packages-select lab2_nodes
source install/setup.bash
Starting >>> lab2_nodes
[Processing: lab2_nodes]
Finished <<< lab2_nodes [44.3s]
Summary: 1 package finished [44.7s]
root@8ba1f5e8b1f8:/ros2_ws# ros2 run lab2_nodes data_filter
[INFO] [1791721004.826726822] [data_filter]: data_filter запущено, слухачів
немає, підписку не створено
[INFO] [1791721249.845612948] [data_filter]: Кількість слухачів uppercase_to
pic: 1
[INFO] [1791721249.847093571] [data_filter]: Підписку на input_topic СТВОРЕН
О


root@8ba1f5e8b1f8: /ros2_ws
7 minutes intelligent_moser
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker exec -it 8ba1f5e8
b1f8 bash
root@8ba1f5e8b1f8:/ros2_ws# source /opt/ros/jazzy/setup.bash
source /ros2_ws/install/setup.bash
root@8ba1f5e8b1f8:/ros2_ws# ros2 param list /data_filter
qos_overrides./parameter_events.publisher.depth
qos_overrides./parameter_events.publisher.durability
qos_overrides./parameter_events.publisher.history
qos_overrides./parameter_events.publisher.reliability
qos_overrides./uppercase_topic.publisher.depth
qos_overrides./uppercase_topic.publisher.durability
qos_overrides./uppercase_topic.publisher.reliability
start_type_description_service
use_sim_time
root@8ba1f5e8b1f8:/ros2_ws# ros2 topic echo /uppercase_topic
data: HELLO ROS TWO
---
root@8ba1f5e8b1f8: /ros2_ws
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker ps
CONTAINER ID IMAGE COMMAND CREATED STAT
US PORTS NAMES
8ba1f5e8b1f8 my_ros2_jazzy "/entrypoint.sh bash" 27 minutes ago Up 2
7 minutes intelligent_moser
PS C:\Users\Даня\Desktop\lections _4.1\РПЗСРЧ\lab2> docker exec -it 8ba1f5e8
b1f8 bash
root@8ba1f5e8b1f8:/ros2_ws# source /opt/ros/jazzy/setup.bash
source /ros2_ws/install/setup.bash
root@8ba1f5e8b1f8:/ros2_ws# ros2 topic pub --once /input_topic std_msgs/msg/
String "{data: 'hello ros two'}"
publisher: beginning loop
publishing #1: std_msgs.msg.String(data='hello ros two')
root@8ba1f5e8b1f8:/ros2_ws#
root@8ba1f5e8b1f8: /ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# cd /ros2_ws
colcon build --symlink-install --packages-select lab2_nodes
source install/setup.bash
Starting >>> lab2_nodes
[Processing: lab2_nodes]
Finished <<< lab2_nodes [44.3s]
Summary: 1 package finished [44.7s]
root@8ba1f5e8b1f8:/ros2_ws# ros2 run lab2_nodes data_filter
[INFO] [1791721004.826726822] [data_filter]: data_filter запущено, слухачів
немає, підписку не створено
[INFO] [1791721249.845612948] [data_filter]: Кількість слухачів uppercase_to
pic: 1
[INFO] [1791721249.847093571] [data_filter]: Підписку на input_topic СТВОРЕН
О






```cpp
#include <algorithm>
#include <cctype>
#include <chrono>
#include <memory>
#include <string>
#include <thread>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"
#include "lab2_interfaces/srv/reset_filter.hpp"

using ResetFilter = lab2_interfaces::srv::ResetFilter;

class DataFilter : public rclcpp::Node
{
public:
```

```
  DataFilter()
  : Node("data_filter")
  {
    rclcpp::PublisherOptions pub_options;

    pub_options.qos_overriding_options = rclcpp::QosOverridingOptions(
      {rclcpp::QosPolicyKind::Depth,
        rclcpp::QosPolicyKind::Reliability,
        rclcpp::QosPolicyKind::Durability});

    pub_options.event_callbacks.matched_callback =
      [this](auto & status) {
        on_matched(static_cast<int>(status.current_count));
      };

    auto qos = rclcpp::QoS(rclcpp::KeepLast(1)).transient_local();
    pub_ = create_publisher<std_msgs::msg::String>("uppercase_topic", qos,
pub_options);

    // П.5: сервіс ResetFilter
    declare_parameter<double>("reset_delay_sec", 0.0);
    service_ = create_service<ResetFilter>(
      "reset_filter",
      [this](
        const std::shared_ptr<ResetFilter::Request>,
        std::shared_ptr<ResetFilter::Response> response) {
        on_reset(response);
      });

    // П.5: інтроспекція сервісу зі станом CONTENTS (метадані + вміст
запитів/відповідей)
    service_->configure_introspection(
      get_clock(), rclcpp::SystemDefaultsQoS(),
RCL_SERVICE_INTROSPECTION_CONTENTS);

    RCLCPP_INFO(get_logger(), "data_filter запущено, слухачів немає,
підписку не створено");
  }

private:
  void on_matched(int listeners)
  {
    RCLCPP_INFO(get_logger(), "Кількість слухачів uppercase_topic: %d",
listeners);

    if (listeners > 0 && !sub_) {
      sub_ = create_subscription<std_msgs::msg::String>(
        "input_topic", 10,
        [this](const std_msgs::msg::String & msg) {on_input(msg);});
      RCLCPP_INFO(get_logger(), "Підписку на input_topic СТВОРЕНО");
```

```
    } else if (listeners == 0 && sub_) {
      sub_.reset();
      RCLCPP_INFO(get_logger(), "Підписку на input_topic ВИМКНЕНО");
    }
  }

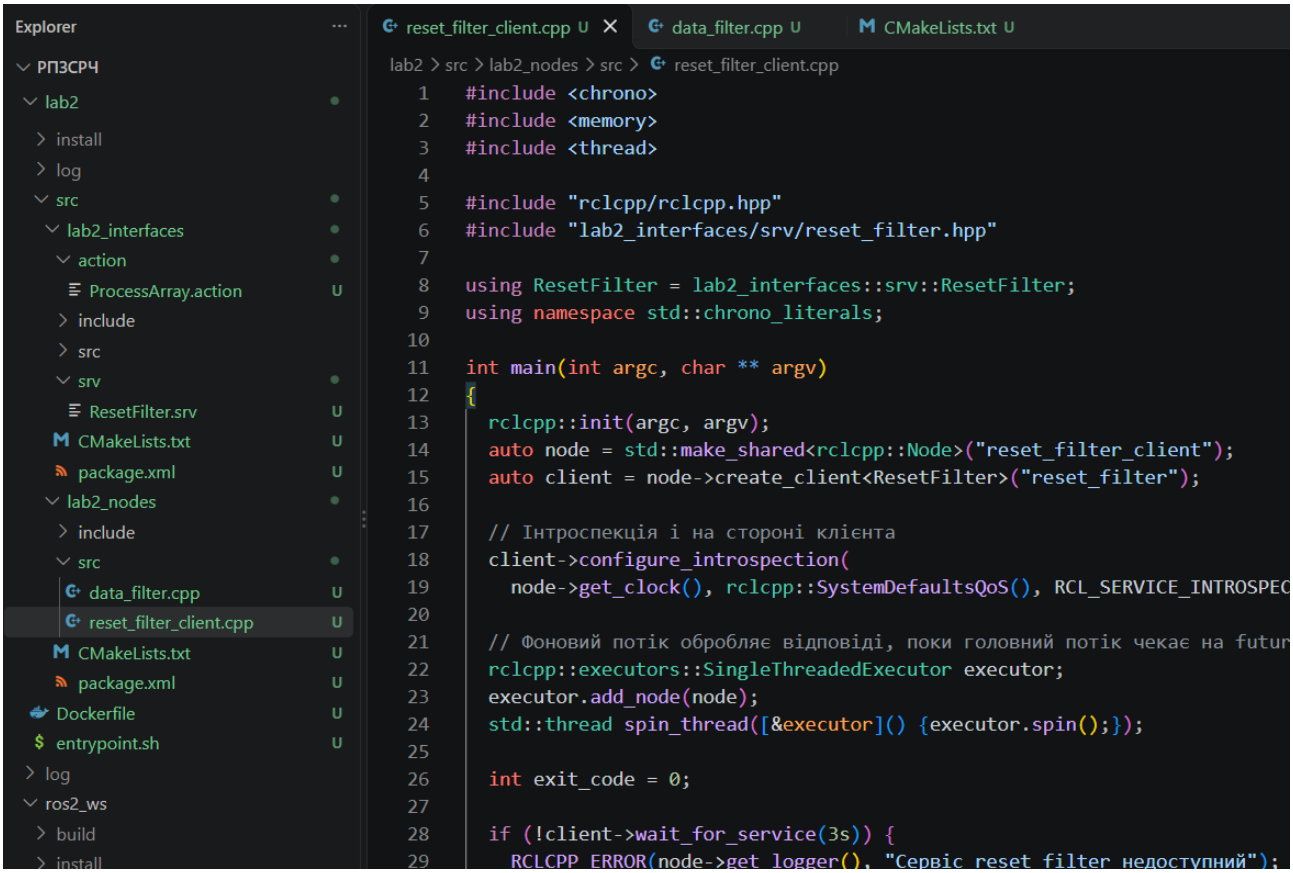
  void on_input(const std_msgs::msg::String & msg)
  {
    std_msgs::msg::String out;
    out.data = msg.data;
    std::transform(
      out.data.begin(), out.data.end(), out.data.begin(),
      [](unsigned char c) {return static_cast<char>(std::toupper(c));});
    pub_->publish(out);
    ++processed_count_;
  }

  void on_reset(std::shared_ptr<ResetFilter::Response> response)
  {
    // Штучна затримка (за замовчуванням 0), щоб перевірити таймаут клієнта
    double delay = get_parameter("reset_delay_sec").as_double();
    if (delay > 0.0) {
      std::this_thread::sleep_for(std::chrono::duration<double>(delay));
    }

    response->success = true;
    response->message = "Фільтр скинуто, було оброблено повідомлень: " +
      std::to_string(processed_count_);
    processed_count_ = 0;
    RCLCPP_INFO(get_logger(), "ResetFilter: %s", response->message.c_str());
  }

  rclcpp::Publisher<std_msgs::msg::String>::SharedPtr pub_;
  rclcpp::Subscription<std_msgs::msg::String>::SharedPtr sub_;
  rclcpp::Service<ResetFilter>::SharedPtr service_;
  uint64_t processed_count_{0};
};

int main(int argc, char ** argv)
{
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<DataFilter>());
  rclcpp::shutdown();
  return 0;
}
```




```cpp
#include <chrono>
#include <memory>
#include <thread>

#include "rclcpp/rclcpp.hpp"
#include "lab2_interfaces/srv/reset_filter.hpp"

using ResetFilter = lab2_interfaces::srv::ResetFilter;
using namespace std::chrono_literals;

int main(int argc, char ** argv)
{
  rclcpp::init(argc, argv);
  auto node = std::make_shared<rclcpp::Node>("reset_filter_client");
  auto client = node->create_client<ResetFilter>("reset_filter");

  // Інтроспекція і на стороні клієнта
  client->configure_introspection(
    node->get_clock(), rclcpp::SystemDefaultsQoS(),
RCL_SERVICE_INTROSPECTION_CONTENTS);

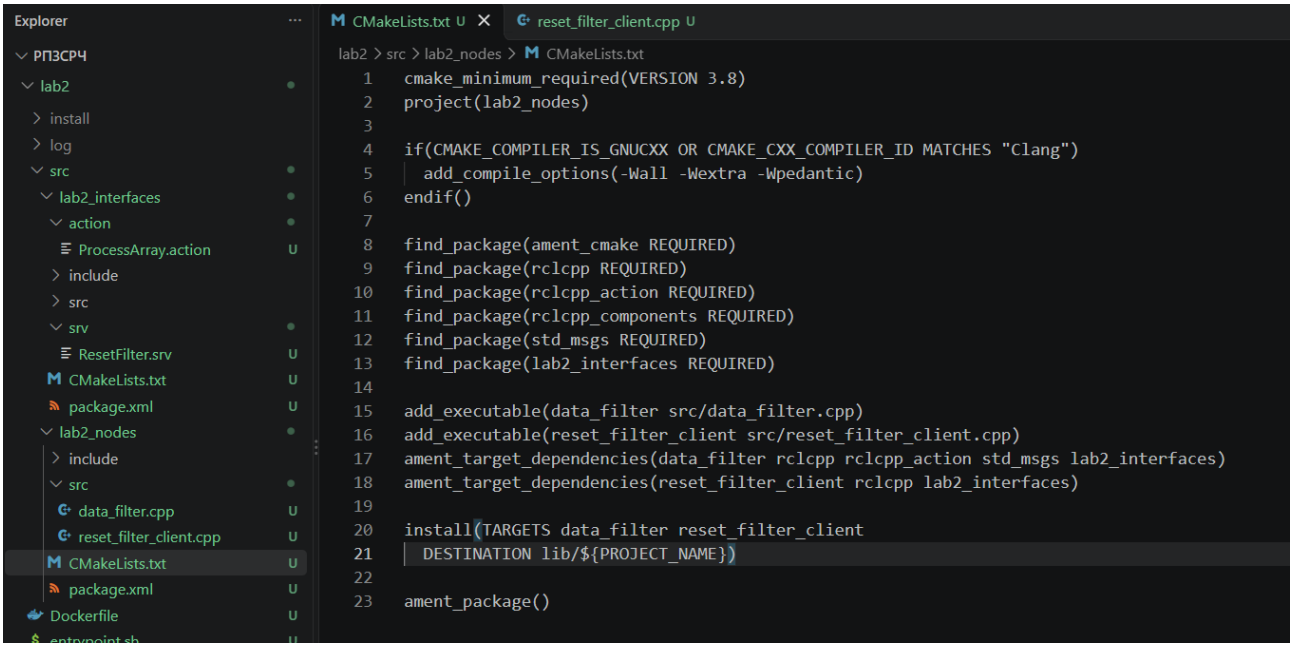
  // Фоновий потік обробляє відповіді, поки головний потік чекає на future
  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(node);
  std::thread spin_thread([&executor]() {executor.spin();});

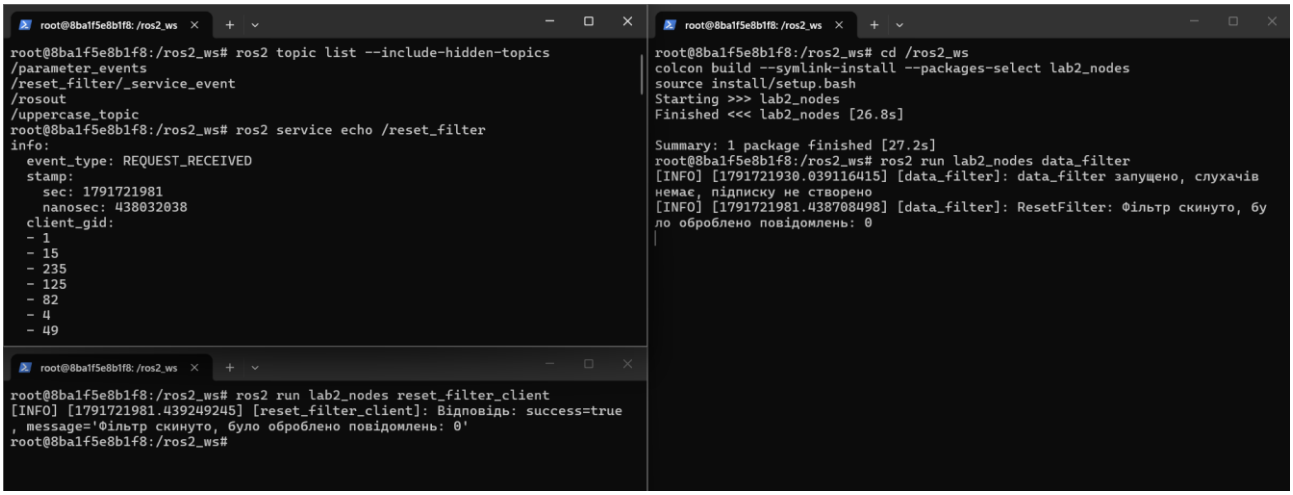
  int exit_code = 0;
```

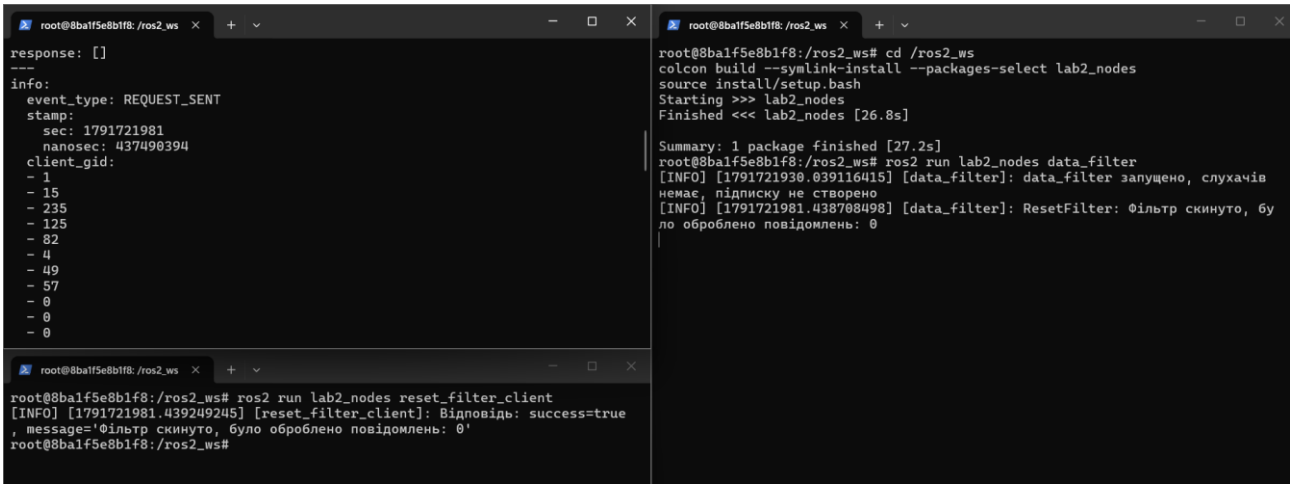
```
  if (!client->wait_for_service(3s)) {
    RCLCPP_ERROR(node->get_logger(), "Сервіс reset_filter недоступний");
    exit_code = 1;
  } else {
    auto request = std::make_shared<ResetFilter::Request>();
    auto result = client->async_send_request(request);

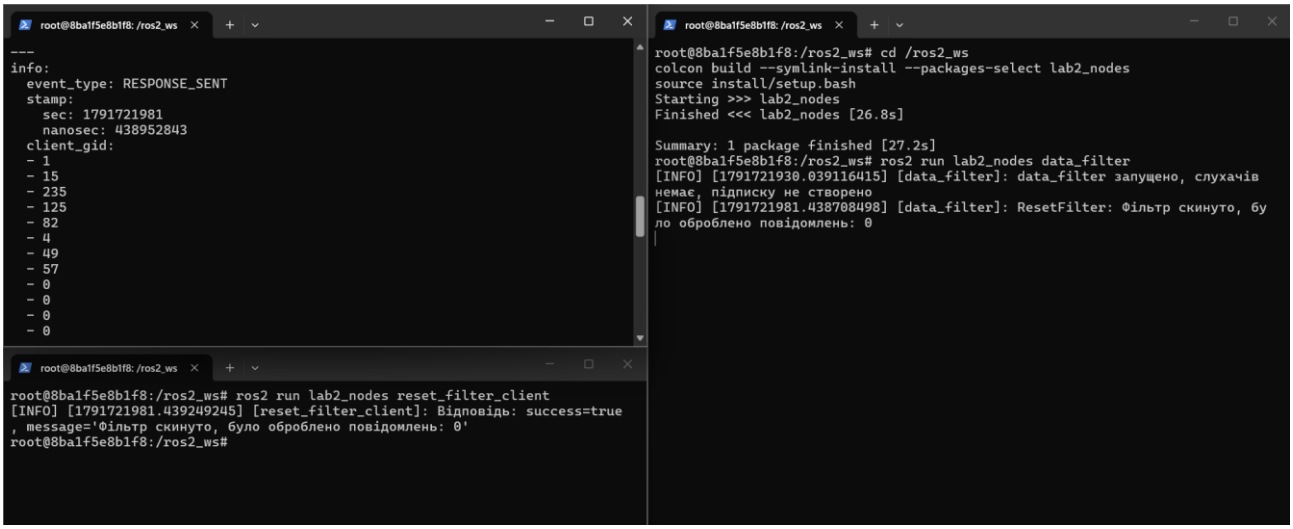
    // Обробка таймауту очікування відповіді
    if (result.future.wait_for(5s) == std::future_status::ready) {
      auto response = result.future.get();
      RCLCPP_INFO(
        node->get_logger(), "Відповідь: success=%s, message='%s'",
        response->success ? "true" : "false", response->message.c_str());
    } else {
      RCLCPP_ERROR(node->get_logger(), "Таймаут: відповіді не отримано за 5
секунд");
      client->remove_pending_request(result.request_id);
      exit_code = 2;
    }
  }

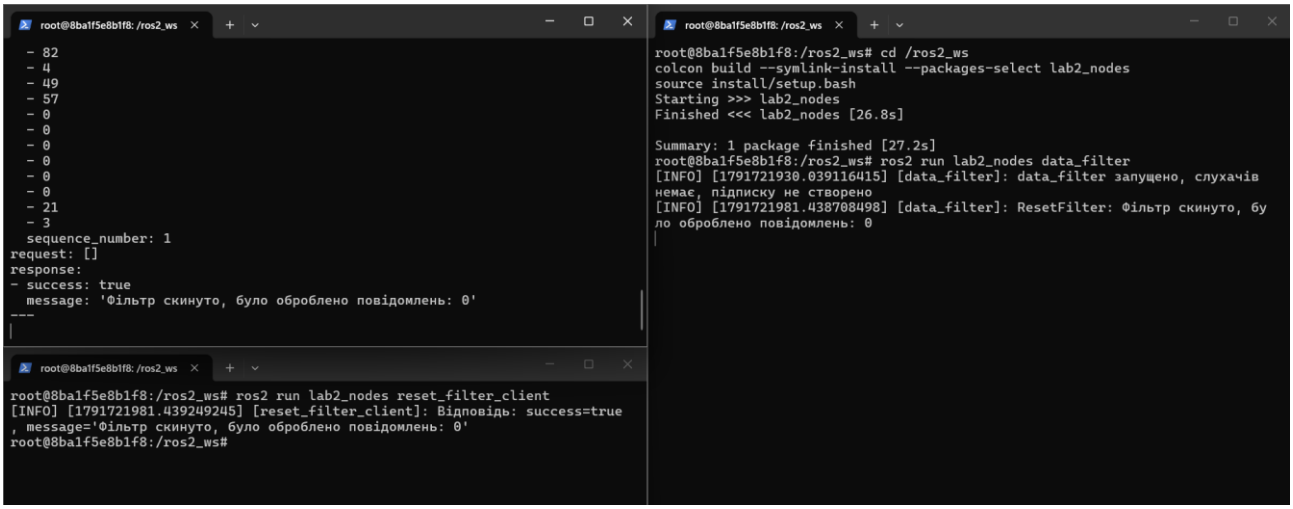
  executor.cancel();
  spin_thread.join();
  rclcpp::shutdown();
  return exit_code;
}
```

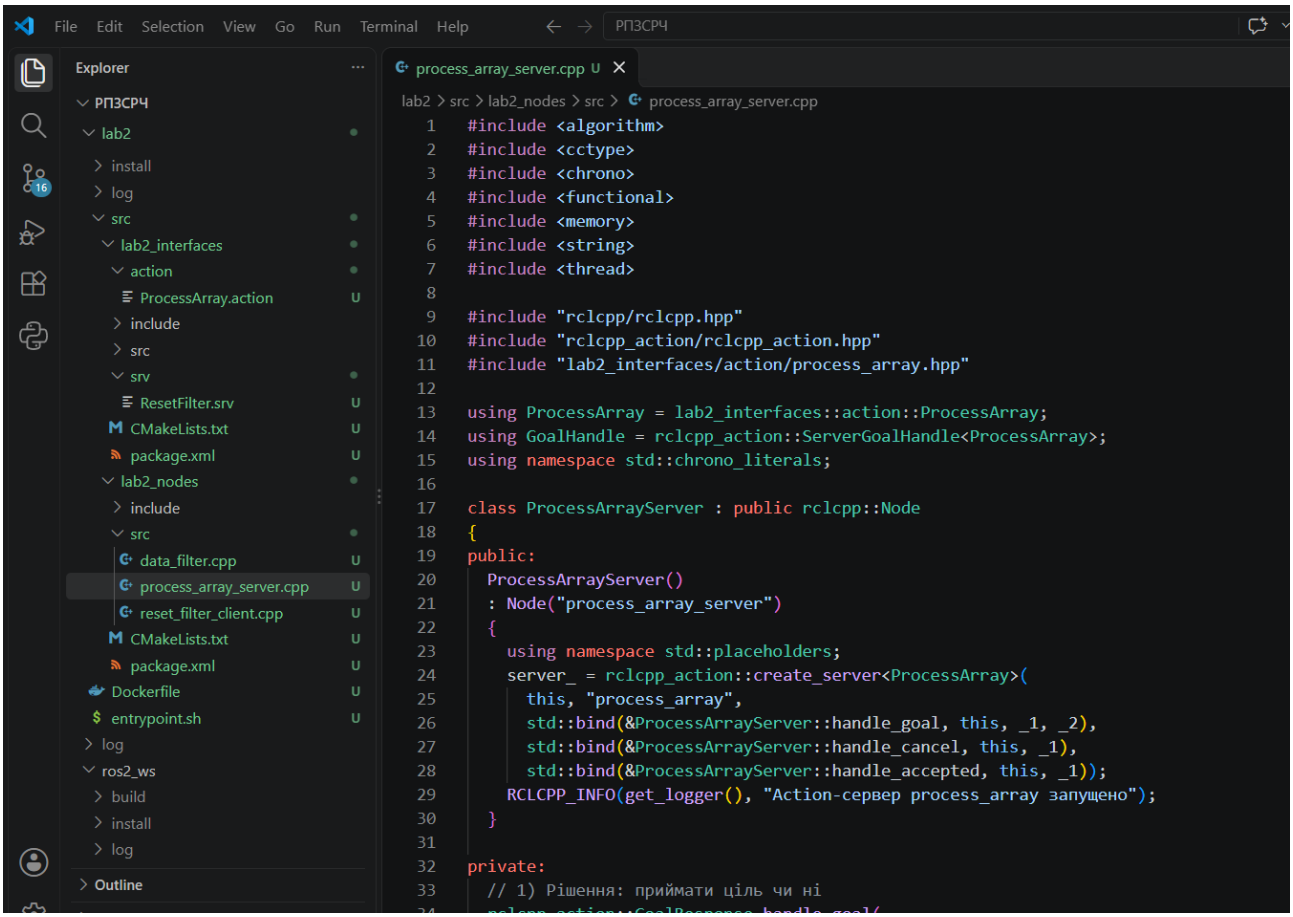



root@8ba1f5e8b1f8: /ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# ros2 topic list --include-hidden-topics
/parameter_events
/reset_filter/_service_event
/rosout
/uppercase_topic
root@8ba1f5e8b1f8:/ros2_ws# ros2 service echo /reset_filter
info:
event_type: REQUEST_RECEIVED
stamp:
sec: 1791721981
nanosec: 438032038
client_gid:
- 1
- 15
- 235
- 125
- 82
- 4
- 49
root@8ba1f5e8b1f8:/ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# ros2 run lab2_nodes reset_filter_client
[INFO] [1791721981.439249245] [reset_filter_client]: Відповідь: success=true
, message='Фільтр скинуто, було оброблено повідомлень: 0'
root@8ba1f5e8b1f8:/ros2_ws#
root@8ba1f5e8b1f8: /ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# cd /ros2_ws
colcon build --symlink-install --packages-select lab2_nodes
source install/setup.bash
Starting >>> lab2_nodes
Finished <<< lab2_nodes [26.8s]
Summary: 1 package finished [27.2s]
root@8ba1f5e8b1f8:/ros2_ws# ros2 run lab2_nodes data_filter
[INFO] [1791721930.039116415] [data_filter]: data_filter запущено, слухачів
немає, підписку не створено
[INFO] [1791721981.438708498] [data_filter]: ResetFilter: Фільтр скинуто, бу
ло оброблено повідомлень: 0


root@8ba1f5e8b1f8: /ros2_ws
response: []
---
info:
event_type: REQUEST_SENT
stamp:
sec: 1791721981
nanosec: 437490394
client_gid:
- 1
- 15
- 235
- 125
- 82
- 4
- 49
- 57
- 0
- 0
- 0
root@8ba1f5e8b1f8:/ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# ros2 run lab2_nodes reset_filter_client
[INFO] [1791721981.439249245] [reset_filter_client]: Відповідь: success=true
, message='Фільтр скинуто, було оброблено повідомлень: 0'
root@8ba1f5e8b1f8:/ros2_ws#
root@8ba1f5e8b1f8: /ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# cd /ros2_ws
colcon build --symlink-install --packages-select lab2_nodes
source install/setup.bash
Starting >>> lab2_nodes
Finished <<< lab2_nodes [26.8s]
Summary: 1 package finished [27.2s]
root@8ba1f5e8b1f8:/ros2_ws# ros2 run lab2_nodes data_filter
[INFO] [1791721930.039116415] [data_filter]: data_filter запущено, слухачів
немає, підписку не створено
[INFO] [1791721981.438708498] [data_filter]: ResetFilter: Фільтр скинуто, бу
ло оброблено повідомлень: 0


root@8ba1f5e8b1f8: /ros2_ws
---
info:
event_type: RESPONSE_SENT
stamp:
sec: 1791721981
nanosec: 438952843
client_gid:
- 1
- 15
- 235
- 125
- 82
- 4
- 49
- 57
- 0
- 0
- 0
- 0
root@8ba1f5e8b1f8:/ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# ros2 run lab2_nodes reset_filter_client
[INFO] [1791721981.439249245] [reset_filter_client]: Відповідь: success=true
, message='Фільтр скинуто, було оброблено повідомлень: 0'
root@8ba1f5e8b1f8:/ros2_ws#
root@8ba1f5e8b1f8: /ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# cd /ros2_ws
colcon build --symlink-install --packages-select lab2_nodes
source install/setup.bash
Starting >>> lab2_nodes
Finished <<< lab2_nodes [26.8s]
Summary: 1 package finished [27.2s]
root@8ba1f5e8b1f8:/ros2_ws# ros2 run lab2_nodes data_filter
[INFO] [1791721930.039116415] [data_filter]: data_filter запущено, слухачів
немає, підписку не створено
[INFO] [1791721981.438708498] [data_filter]: ResetFilter: Фільтр скинуто, бу
ло оброблено повідомлень: 0






```cpp
#include <algorithm>
#include <cctype>
#include <chrono>
#include <functional>
#include <memory>
#include <string>
#include <thread>

#include "rclcpp/rclcpp.hpp"
#include "rclcpp_action/rclcpp_action.hpp"
#include "lab2_interfaces/action/process_array.hpp"

using ProcessArray = lab2_interfaces::action::ProcessArray;
```

```
using GoalHandle = rclcpp_action::ServerGoalHandle<ProcessArray>;
using namespace std::chrono_literals;

class ProcessArrayServer : public rclcpp::Node
{
public:
  ProcessArrayServer()
  : Node("process_array_server")
  {
    using namespace std::placeholders;
    server_ = rclcpp_action::create_server<ProcessArray>(
      this, "process_array",
      std::bind(&ProcessArrayServer::handle_goal, this, _1, _2),
      std::bind(&ProcessArrayServer::handle_cancel, this, _1),
      std::bind(&ProcessArrayServer::handle_accepted, this, _1));
    RCLCPP_INFO(get_logger(), "Action-сервер process_array запущено");
  }

private:
  // 1) Рішення: приймати ціль чи ні
  rclcpp_action::GoalResponse handle_goal(
    const rclcpp_action::GoalUUID &,
    std::shared_ptr<const ProcessArray::Goal> goal)
  {
    if (goal->data.empty()) {
      RCLCPP_WARN(get_logger(), "Ціль відхилено: порожній масив");
      return rclcpp_action::GoalResponse::REJECT;
    }
    RCLCPP_INFO(get_logger(), "Ціль прийнято, елементів: %zu", goal-
>data.size());
    return rclcpp_action::GoalResponse::ACCEPT_AND_EXECUTE;
  }

  // 2) Рішення: чи дозволяти скасування
  rclcpp_action::CancelResponse handle_cancel(const
std::shared_ptr<GoalHandle>)
  {
    RCLCPP_INFO(get_logger(), "Отримано запит на скасування");
    return rclcpp_action::CancelResponse::ACCEPT;
  }

  // 3) Ціль прийнято: запускаємо виконання в ОКРЕМОМУ потоці,
  //    щоб не блокувати executor
  void handle_accepted(const std::shared_ptr<GoalHandle> goal_handle)
  {
    std::thread{
      std::bind(&ProcessArrayServer::execute, this, std::placeholders::_1),
      goal_handle}.detach();
  }
```

```cpp
  void execute(const std::shared_ptr<GoalHandle> goal_handle)
  {
    const auto goal = goal_handle->get_goal();
    auto feedback = std::make_shared<ProcessArray::Feedback>();
    auto result = std::make_shared<ProcessArray::Result>();

    const uint32_t total = static_cast<uint32_t>(goal->data.size());
    feedback->total = total;

    for (uint32_t i = 0; i < total; ++i) {
      // Перевірка скасування на кожній ітерації
      if (goal_handle->is_canceling()) {
        result->success = false;
        goal_handle->canceled(result);
        RCLCPP_INFO(get_logger(), "Ціль скасовано на елементі %u з %u", i,
total);
        return;
      }

      std::string item = goal->data[i];
      std::transform(
        item.begin(), item.end(), item.begin(),
        [](unsigned char c) {return static_cast<char>(std::toupper(c));});
      result->processed.push_back(item);

      // Періодичний feedback про поточний стан
      feedback->processed_count = i + 1;
      feedback->state = "Processing";
      goal_handle->publish_feedback(feedback);

      std::this_thread::sleep_for(500ms);  // імітація тривалої обробки
    }

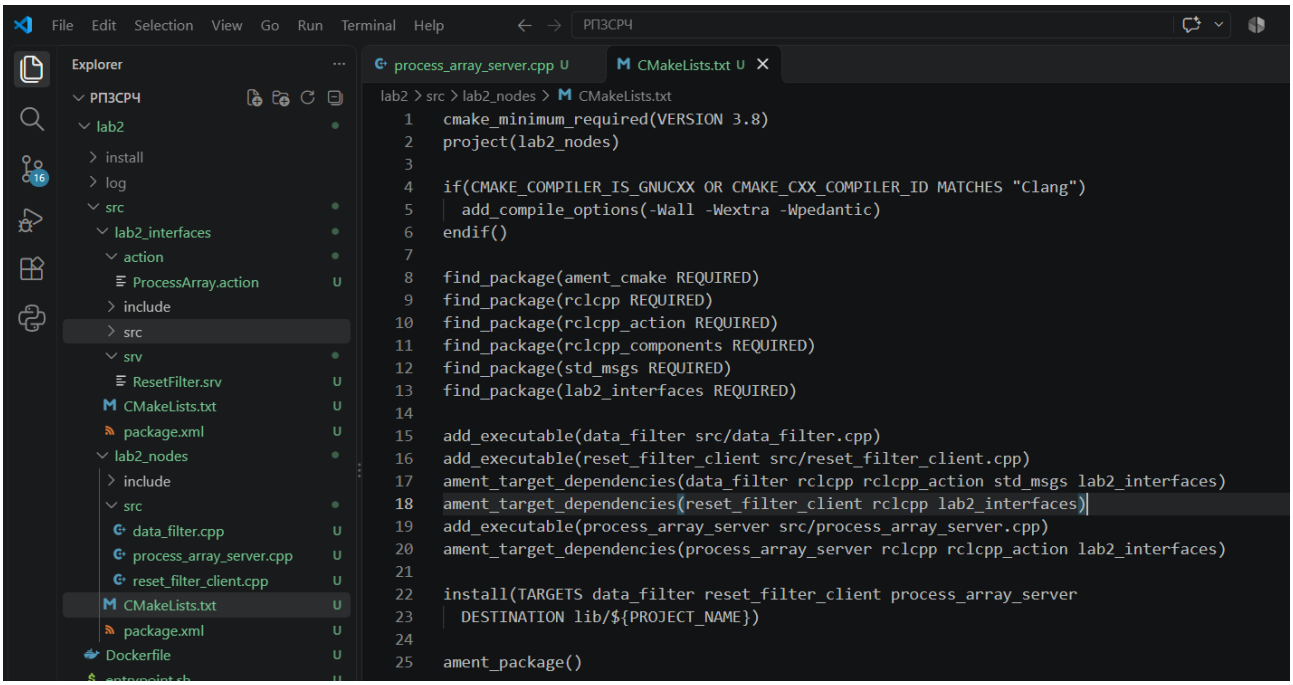
    // Фінальний feedback зі станом Succeeded, потім результат
    feedback->state = "Succeeded";
    goal_handle->publish_feedback(feedback);

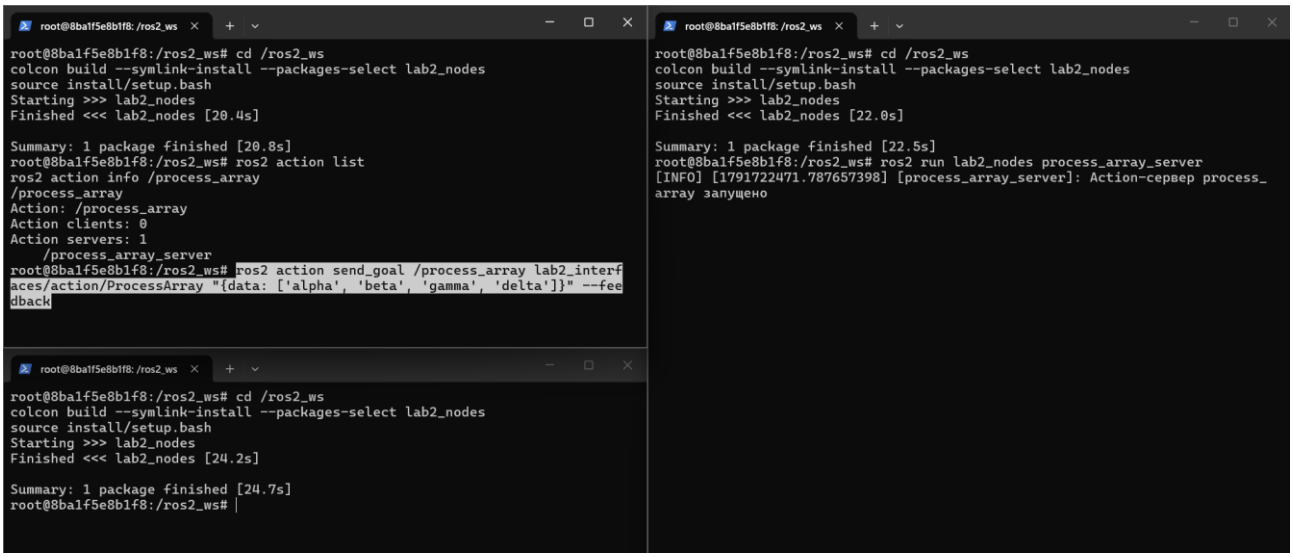
    result->success = true;
    goal_handle->succeed(result);
    RCLCPP_INFO(get_logger(), "Ціль виконано успішно");
  }

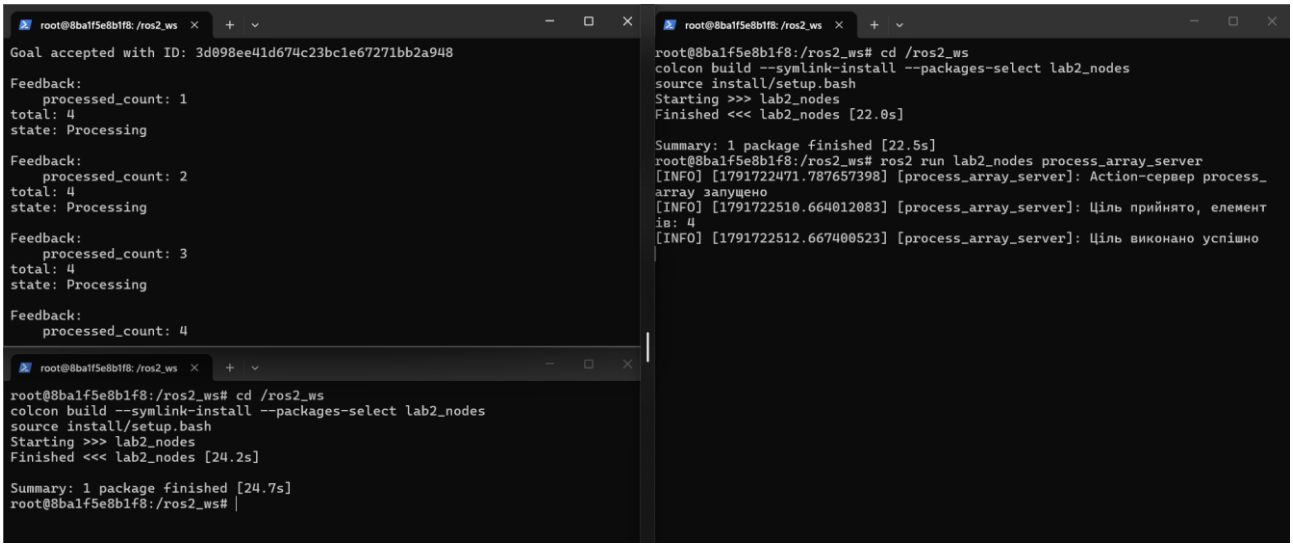
  rclcpp_action::Server<ProcessArray>::SharedPtr server_;
};

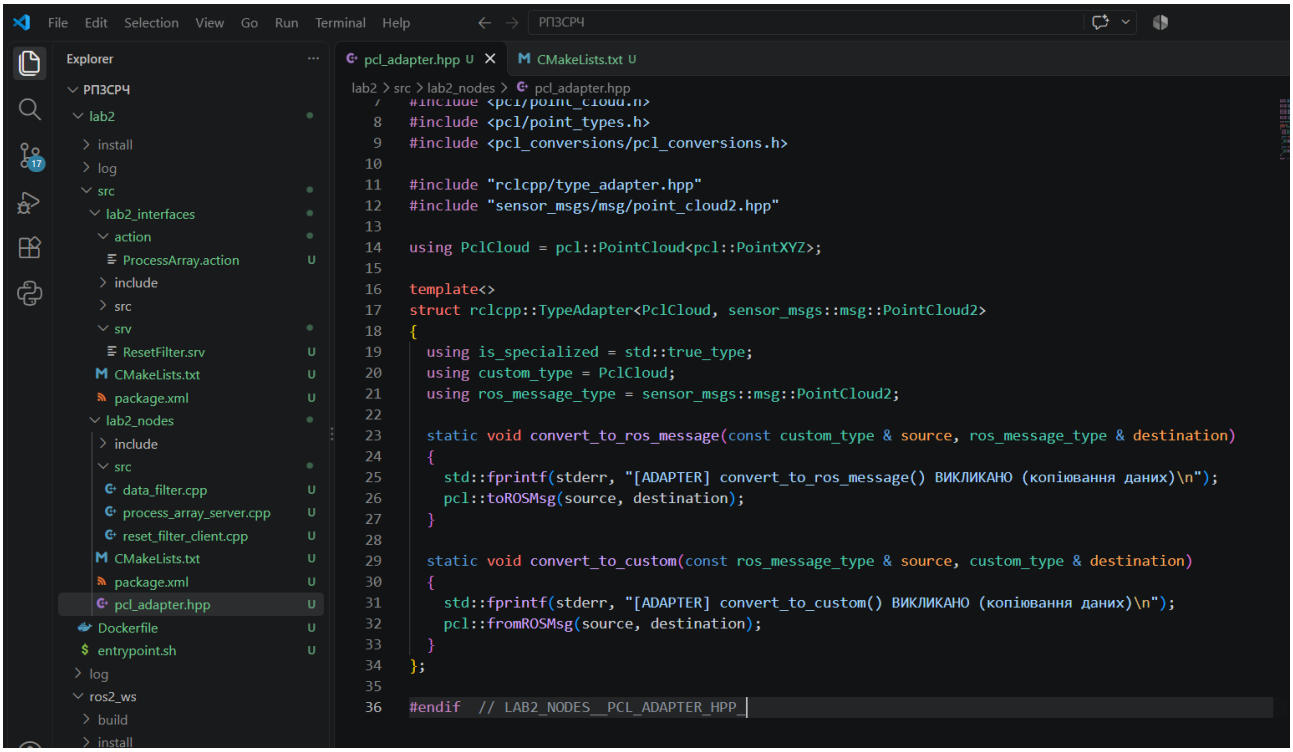
int main(int argc, char ** argv)
{
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<ProcessArrayServer>());
  rclcpp::shutdown();
```

```
  return 0;
}
```


File Edit Selection View Go Run Terminal Help
РПЗСРЧ
Explorer
process_array_server.cpp U
CMakeLists.txt U
lab2 > src > lab2_nodes > CMakeLists.txt
cmake_minimum_required(VERSION 3.8)
project(lab2_nodes)
if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
add_compile_options(-Wall -Wextra -Wpedantic)
endif()
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(rclcpp_action REQUIRED)
find_package(rclcpp_components REQUIRED)
find_package(std_msgs REQUIRED)
find_package(lab2_interfaces REQUIRED)
add_executable(data_filter src/data_filter.cpp)
add_executable(reset_filter_client src/reset_filter_client.cpp)
ament_target_dependencies(data_filter rclcpp rclcpp_action std_msgs lab2_interfaces)
ament_target_dependencies(reset_filter_client rclcpp lab2_interfaces)
add_executable(process_array_server src/process_array_server.cpp)
ament_target_dependencies(process_array_server rclcpp rclcpp_action lab2_interfaces)
install(TARGETS data_filter reset_filter_client process_array_server
DESTINATION lib/${PROJECT_NAME})
ament_package()


root@8ba1f5e8b1f8: /ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# cd /ros2_ws
colcon build --symlink-install --packages-select lab2_nodes
source install/setup.bash
Starting >>> lab2_nodes
Finished <<< lab2_nodes [20.4s]
Summary: 1 package finished [20.8s]
root@8ba1f5e8b1f8:/ros2_ws# ros2 action list
ros2 action info /process_array
/process_array
Action: /process_array
Action clients: 0
Action servers: 1
/process_array_server
root@8ba1f5e8b1f8:/ros2_ws# ros2 action send_goal /process_array lab2_interfaces/action/ProcessArray "{data: ['alpha', 'beta', 'gamma', 'delta']}" --feedback
root@8ba1f5e8b1f8: /ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# cd /ros2_ws
colcon build --symlink-install --packages-select lab2_nodes
source install/setup.bash
Starting >>> lab2_nodes
Finished <<< lab2_nodes [24.2s]
Summary: 1 package finished [24.7s]
root@8ba1f5e8b1f8:/ros2_ws#
root@8ba1f5e8b1f8: /ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# cd /ros2_ws
colcon build --symlink-install --packages-select lab2_nodes
source install/setup.bash
Starting >>> lab2_nodes
Finished <<< lab2_nodes [22.0s]
Summary: 1 package finished [22.5s]
root@8ba1f5e8b1f8:/ros2_ws# ros2 run lab2_nodes process_array_server
[INFO] [1791722471.787657398] [process_array_server]: Action-сервер process_array запущено


root@8ba1f5e8b1f8: /ros2_ws
Goal accepted with ID: 3d098ee41d674c23bc1e67271bb2a948
Feedback:
processed_count: 1
total: 4
state: Processing
Feedback:
processed_count: 2
total: 4
state: Processing
Feedback:
processed_count: 3
total: 4
state: Processing
Feedback:
processed_count: 4
root@8ba1f5e8b1f8:/ros2_ws# cd /ros2_ws
colcon build --symlink-install --packages-select lab2_nodes
source install/setup.bash
Starting >>> lab2_nodes
Finished <<< lab2_nodes [24.2s]
Summary: 1 package finished [24.7s]
root@8ba1f5e8b1f8:/ros2_ws#
root@8ba1f5e8b1f8: /ros2_ws
root@8ba1f5e8b1f8:/ros2_ws# cd /ros2_ws
colcon build --symlink-install --packages-select lab2_nodes
source install/setup.bash
Starting >>> lab2_nodes
Finished <<< lab2_nodes [22.0s]
Summary: 1 package finished [22.5s]
root@8ba1f5e8b1f8:/ros2_ws# ros2 run lab2_nodes process_array_server
[INFO] [1791722471.787657398] [process_array_server]: Action-сервер process_array запущено
[INFO] [1791722510.664012083] [process_array_server]: Ціль прийнято, елементів: 4
[INFO] [1791722512.667400523] [process_array_server]: Ціль виконано успішно




```cpp
#ifndef LAB2_NODES__PCL_ADAPTER_HPP_
#define LAB2_NODES__PCL_ADAPTER_HPP_

#include <cstdio>
#include <type_traits>

#include <pcl/point_cloud.h>
#include <pcl/point_types.h>
#include <pcl_conversions/pcl_conversions.h>

#include "rclcpp/type_adapter.hpp"
#include "sensor_msgs/msg/point_cloud2.hpp"

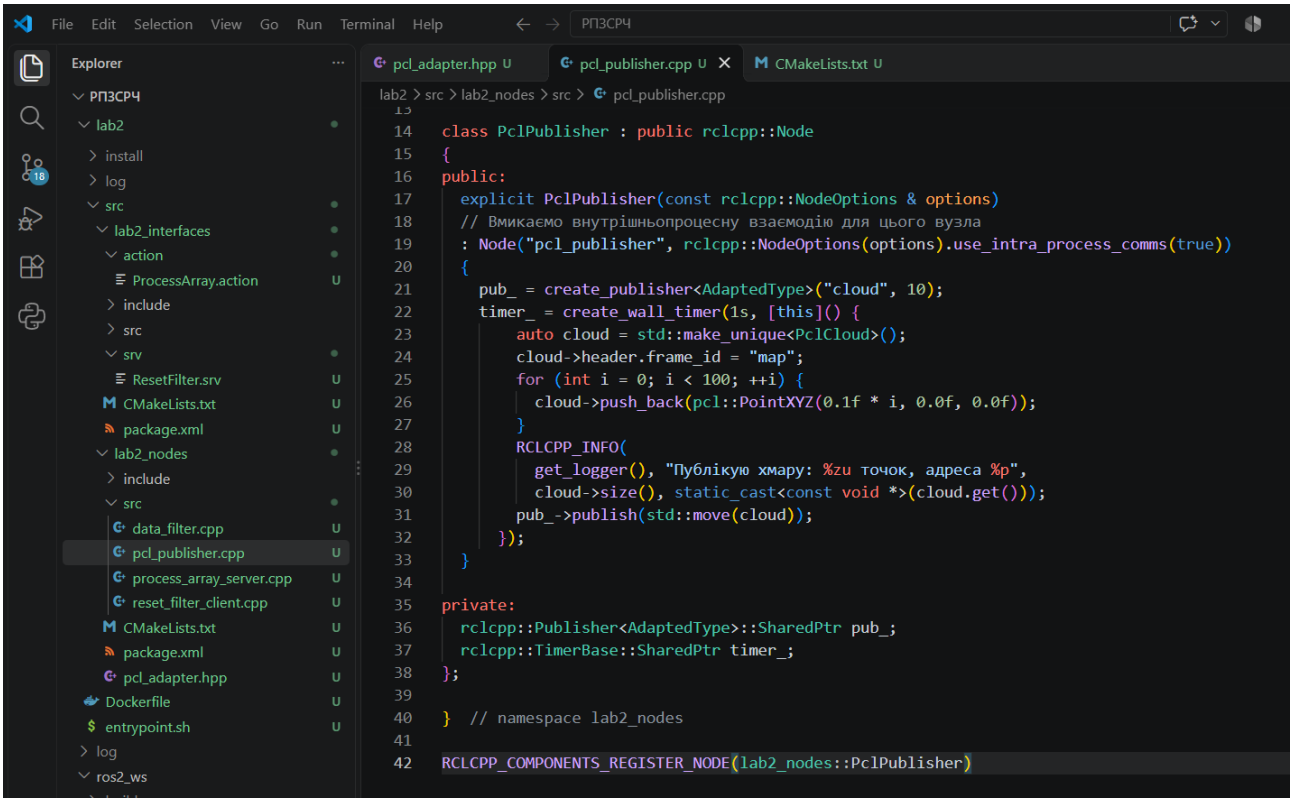
using PclCloud = pcl::PointCloud<pcl::PointXYZ>;

template<>
struct rclcpp::TypeAdapter<PclCloud, sensor_msgs::msg::PointCloud2>
{
  using is_specialized = std::true_type;
  using custom_type = PclCloud;
  using ros_message_type = sensor_msgs::msg::PointCloud2;

  static void convert_to_ros_message(const custom_type & source,
ros_message_type & destination)
  {
    std::fprintf(stderr, "[ADAPTER] convert_to_ros_message() ВИКЛИКАНО
(копіювання даних)\n");
    pcl::toROSMsg(source, destination);
  }
```

```
  static void convert_to_custom(const ros_message_type & source, custom_type
& destination)
  {
    std::fprintf(stderr, "[ADAPTER] convert_to_custom() ВИКЛИКАНО
(копіювання даних)\n");
    pcl::fromROSMsg(source, destination);
  }
};

#endif  // LAB2_NODES__PCL_ADAPTER_HPP_
```




```
#include <chrono>
#include <memory>

#include "lab2_nodes/pcl_adapter.hpp"
#include "rclcpp/rclcpp.hpp"
#include "rclcpp_components/register_node_macro.hpp"

using namespace std::chrono_literals;

namespace lab2_nodes
{
using AdaptedType = rclcpp::TypeAdapter<PclCloud,
sensor_msgs::msg::PointCloud2>;

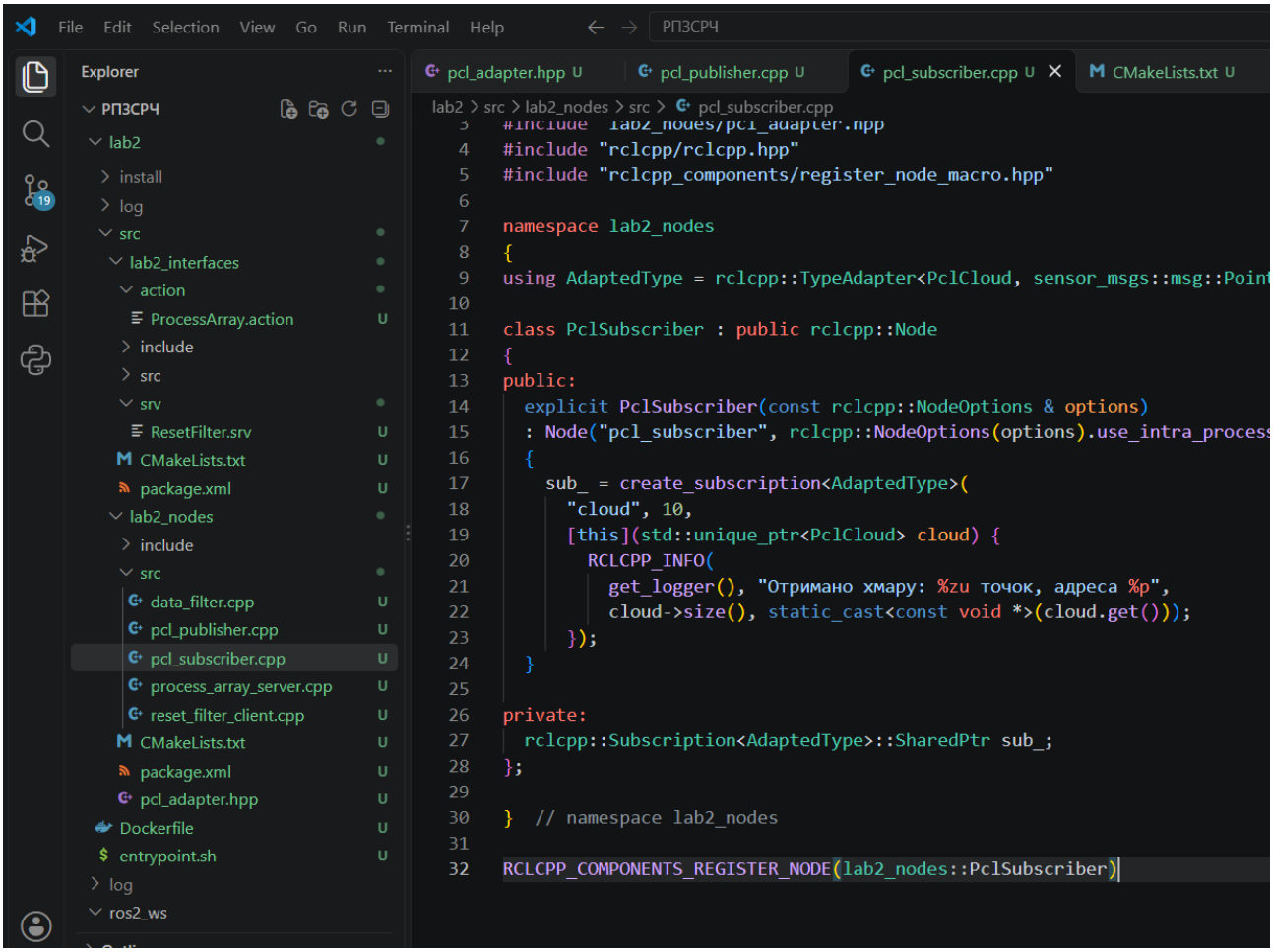
class PclPublisher : public rclcpp::Node
{
public:
  explicit PclPublisher(const rclcpp::NodeOptions & options)
```

```
  // Вмикаємо внутрішньопроцесну взаємодію для цього вузла
  : Node("pcl_publisher",
rclcpp::NodeOptions(options).use_intra_process_comms(true))
  {
    pub_ = create_publisher<AdaptedType>("cloud", 10);
    timer_ = create_wall_timer(1s, [this]() {
        auto cloud = std::make_unique<PclCloud>();
        cloud->header.frame_id = "map";
        for (int i = 0; i < 100; ++i) {
          cloud->push_back(pcl::PointXYZ(0.1f * i, 0.0f, 0.0f));
        }
        RCLCPP_INFO(
          get_logger(), "Публікую хмару: %zu точок, адреса %p",
          cloud->size(), static_cast<const void *>(cloud.get()));
        pub_->publish(std::move(cloud));
      });
  }

private:
  rclcpp::Publisher<AdaptedType>::SharedPtr pub_;
  rclcpp::TimerBase::SharedPtr timer_;
};

}  // namespace lab2_nodes

RCLCPP_COMPONENTS_REGISTER_NODE(lab2_nodes::PclPublisher)
```




```cpp
#include <memory>

#include "lab2_nodes/pcl_adapter.hpp"
#include "rclcpp/rclcpp.hpp"
#include "rclcpp_components/register_node_macro.hpp"

namespace lab2_nodes
{
using AdaptedType = rclcpp::TypeAdapter<PclCloud,
sensor_msgs::msg::PointCloud2>;

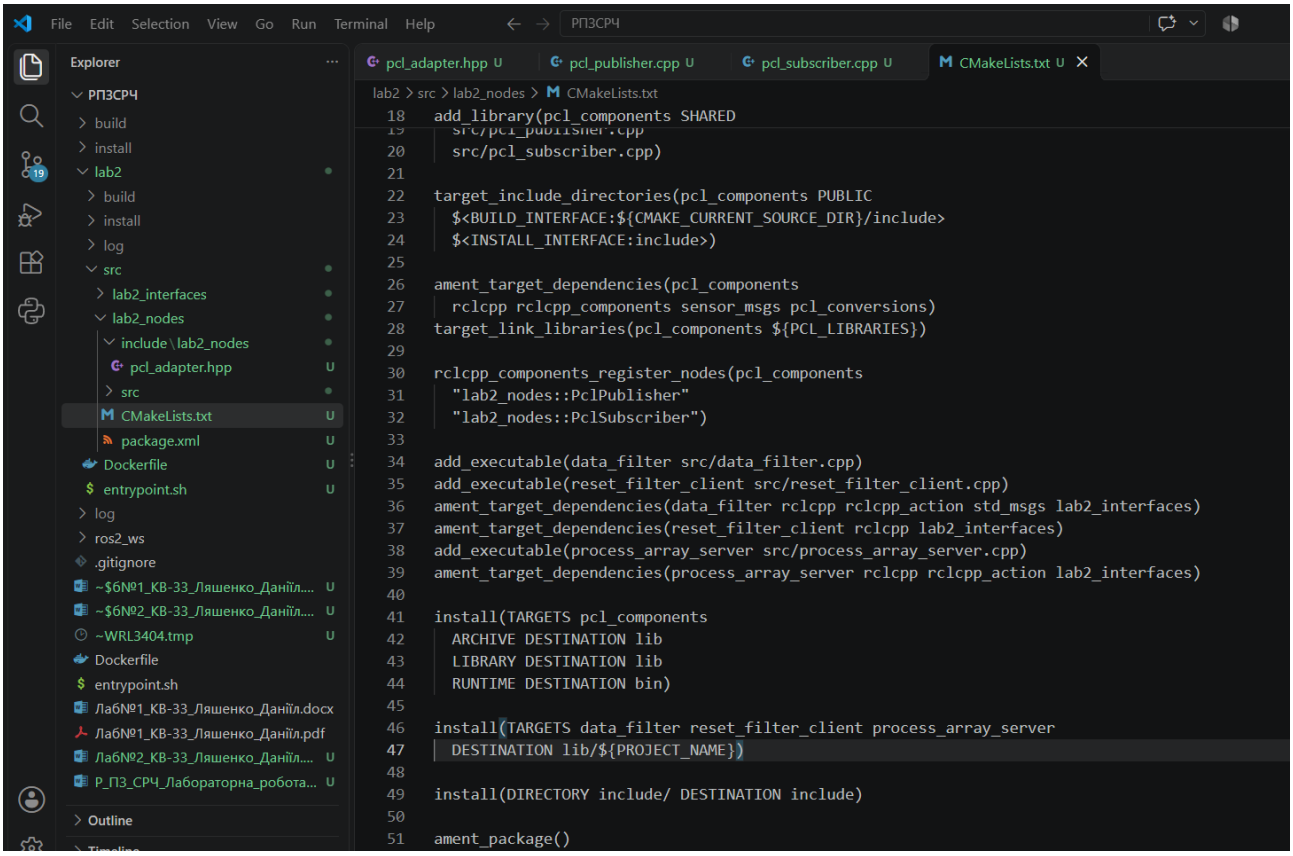
class PclSubscriber : public rclcpp::Node
{
public:
  explicit PclSubscriber(const rclcpp::NodeOptions & options)
  : Node("pcl_subscriber",
rclcpp::NodeOptions(options).use_intra_process_comms(true))
  {
    sub_ = create_subscription<AdaptedType>(
      "cloud", 10,
      [this](std::unique_ptr<PclCloud> cloud) {
        RCLCPP_INFO(
          get_logger(), "Отримано хмару: %zu точок, адреса %p",
          cloud->size(), static_cast<const void *>(cloud.get()));
      });
```

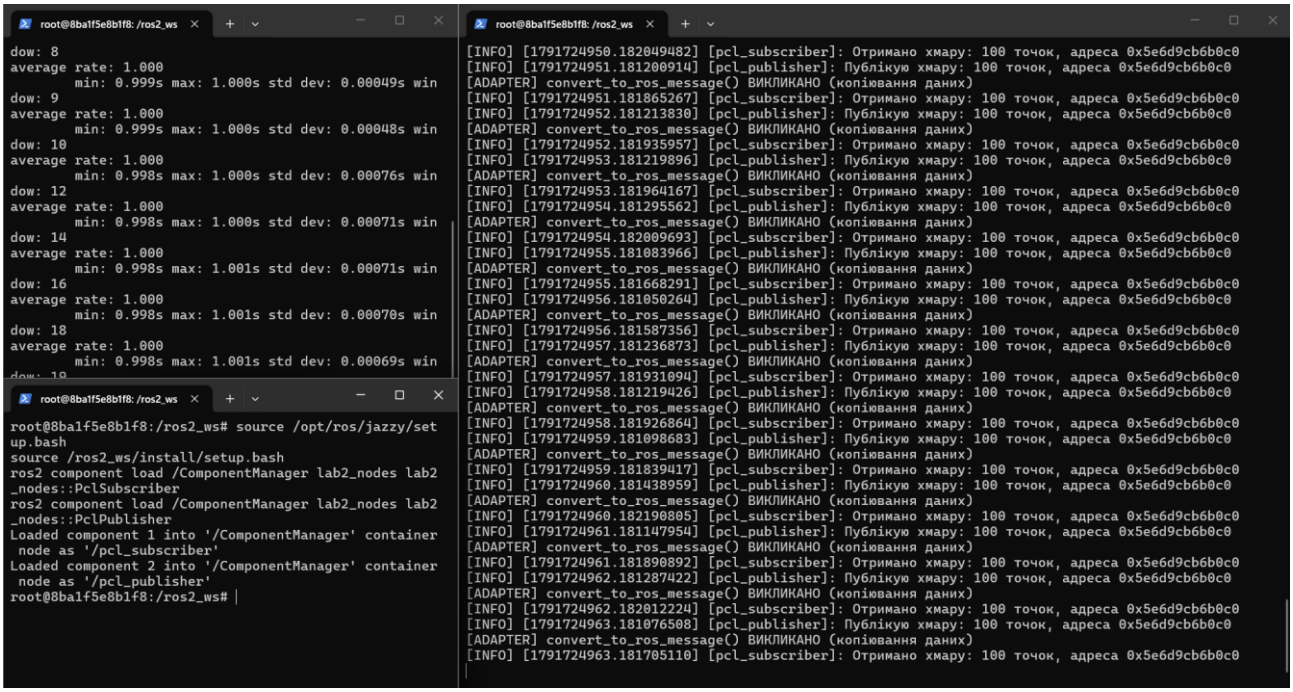
```
  }

private:
  rclcpp::Subscription<AdaptedType>::SharedPtr sub_;
};

}  // namespace lab2_nodes

RCLCPP_COMPONENTS_REGISTER_NODE(lab2_nodes::PclSubscriber)
```





Висновок до лабораторної роботи:

У ході виконання лабораторної роботи було досліджено комунікаційні інтерфейси ROS 2 та механізми керування якістю обслуговування. Для роботи було використано дистрибутив ROS 2 Jazzy, оскільки функції matched_callback та інтроспекції сервісів недоступні в попередньому дистрибутиві Humble. Створено власний пакет інтерфейсів із сервісом ResetFilter та дією ProcessArray, а також реалізовано вузол data_filter, який переводить вхідні рядки у верхній регістр і публікує їх у тему uppercase_topic. Завдяки matched_callback підписка на вхідні дані створюється лише за наявності хоча б одного слухача і видаляється, коли їх не залишається, що економить обчислювальні ресурси. Видавець було налаштовано з профілем QoS transient_local глибиною 1, тому пізній підписник одразу отримує останнє повідомлення, а за допомогою rclcpp::QosOverridingOptions політики Depth, Reliability та Durability можна змінювати через параметри вузла. Реалізовано асинхронний сервіс із клієнтом, що використовує async_send_request та future.wait_for для обробки таймауту, а увімкнена інтроспекція зі станом CONTENTS дозволила перехоплювати події запитів і відповідей через приховану тему _service_event за допомогою команди ros2 service echo. Також розроблено Action-сервер, який виконує обробку масиву в окремому потоці, періодично публікує feedback зі станом виконання та підтримує скасування цілі. Окремо за допомогою rclcpp::TypeAdapter створено адаптер між sensor_msgs::msg::PointCloud2 та pcl::PointCloud, і експериментально підтверджено, що при внутрішньопроцесній взаємодії обидва компоненти працюють з одним і тим самим об'єктом у пам'яті без жодного перетворення, тоді як конвертація в повідомлення ROS виконується лише тоді, коли з'являється слухач в іншому процесі. Отримані результати показали, що поєднання «ледачих» видавців, коректно підібраних політик QoS та адаптації типів дозволяє суттєво знизити навантаження на систему та затримки передачі даних.